

DEZVOLTAREA UNUI SISTEM DE CONTROL DE ZBOR ȘI CAPTURĂ DE IMAGINI AERIENE PENTRU DRONE

Candidat: Norbert KONDORA

Coordonator științific: Dr. Habil. Ing. Mihai V. MICEA

Sesiunea: Septembrie 2026

REZUMAT

Lucrarea de față descrie proiectarea, implementarea unui sistem de control de zbor pentru o dronă construită integral de la zero, precum și al unui subsistem dedicat capturii și salvării de imagini aeriene. Arhitectura hardware a sistemului este împărțită pe două plăci de dezvoltare ESP32-S3 astfel: un controler de zbor, responsabil pentru stabilizarea dronei și un modul de telemetrie care asigură interfața de control utilizată de pilot, printr-o pagina web disponibilă prin Wi-Fi, precum și achiziția, procesarea și salvarea pe un card de memorie a imaginilor capturate cu modulul cameră compatibil. Comunicarea între cele două plăci se realizează printr-un protocol serial simplu iar structura mesajelor este implementată într-un modul software partajat între cele două proiecte ESP-IDF corespunzătoare pentru fiecare dintre cele două module de dezvoltare.

CUPRINS

1	Introducere.....	5
1.1	Contextul temei alese.....	5
1.2	Motivația și scopul proiectului.....	5
1.3	Soluția propusă.....	5
1.4	Contribuția autorului.....	6
2	Soluții similare.....	7
2.1	Sisteme de pilotare automată de uz profesional.....	7
2.2	Firmware pentru drone de mici dimensiuni.....	7
2.3	Soluții bazate pe microcontrolere ESP32.....	8
3	Fundamente teoretice.....	9
3.1	Principiul de zbor al unei drone cu patru motoare.....	9
3.1.1	Subactuarea.....	11
3.1.2	Pragul minim al motoarelor.....	11
3.2	Reglarea automată prin regulator PID.....	12
3.2.1	Componenta proporțională.....	12
3.2.2	Componenta integrală.....	13
3.2.3	Componenta derivativă.....	14
3.2.4	Saturarea și acumularea excesivă.....	16
3.3	Structura de reglare în cascadă.....	16
3.4	Estimarea orientării prin fuziunea senzorilor.....	17
3.4.1	Accelerometrul.....	17
3.4.2	Giroscopul.....	18
3.5	Măsurarea altitudinii și a vitezei de deplasare.....	19
3.5.1	Telemetrul laser.....	19
3.5.2	Senzorul de flux optic.....	19
3.6	Execuția programului în timp real.....	20
3.7	Mediul de dezvoltare și bibliotecile utilizate.....	20
4	Elaborarea proiectului.....	23
4.1	Specificația sistemului.....	23
4.1.1	Cerințe funcționale.....	23
4.1.2	Cerințe nefuncționale.....	23
4.1.3	Constrângeri impuse de hardware.....	25
4.1.4	Delimitarea funcționalităților.....	25
4.2	Arhitectura Software a sistemului.....	26
4.2.1	Împărțirea Sistemului pe două Microcontrolere.....	26
4.2.2	Organizarea pe componente.....	27
4.2.3	Repartizarea sarcinilor pe nuclee și priorități.....	29
4.2.4	Schimbul de date între sarcini.....	30
4.3	Protocolul de comunicație între plăci.....	31
4.3.1	Structura cadrului.....	31
4.3.2	Tipurile de mesaje.....	32
4.3.3	Detectarea pierderii legăturii seriale.....	32
4.3.4	Tratarea erorilor de recepție.....	33
4.4	Controlerul de zbor.....	33
4.4.1	Implementarea regulatorului PID.....	33
4.4.2	Structura în cascadă și cele opt bucle.....	35
4.4.3	Estimatorul de orientare.....	37
4.4.4	Estimatorul de navigație.....	38

4.4.5 Mixerul.....	39
4.4.6 Mașina de stări de armare.....	41
4.4.7 Driverul de motoare.....	42
4.4.8 Monitorizarea acumulatorului.....	43
4.4.9 Ordinea de inițializare.....	43
4.5 Modulul de telemetrie.....	44
4.5.1 Sinteza comenzilor pilotului.....	44
4.5.2 Punctul de acces Wi-Fi.....	45
4.5.3 Serverul HTTP și tabloul de bord.....	45
4.5.4 Subsistemul de capturare a imaginilor.....	46
4.6 Instrumente de analiză și validare.....	47
4.6.1 Stadiul stației de sol.....	47
4.6.2 Traseul datelor de depanare.....	47
4.6.3 Generatorul de semnal de test.....	47
4.6.4 Înregistrarea zborurilor.....	47
4.7 Testarea și verificarea.....	48
4.7.1 Proiecte de calibrare separate.....	48
4.7.2 Modul de monitorizare cu drona dezarmată.....	48
4.7.3 Indicatorii de diagnostic din cadrul de stare.....	49
4.7.4 Modul de banc pentru motoare.....	49
4.7.5 Verificarea frecvență buclei de control.....	49
4.7.6 Ce a rămas neverificat.....	50
5 Descrierea și utilizarea produsului.....	51
5.1 Livrabilele proiectului.....	51
5.2 Punerea în funcțiune.....	51
6 Concluzia.....	53

1 INTRODUCERE

1.1 CONTEXTUL TEMEI ALESE

Dronele au devenit în ultimii ani instrumente obișnuite într-o multitudine de domenii: cartografiere, inspecția clădirilor și a instalațiilor industriale, agricultură, filmare și fotografiere aeriană. Majoritatea acestor aplicații se bazează însă pe un lucru pe care îl luăm ca atare anume, semnalul GPS folosit pentru poziționarea dronei.

În interiorul clădirilor, în apropierea structurilor metalice sau în spații închise, nu ne putem baza pe acest semnal deoarece este imprecis în condițiile de menționate mai sus. Zborul în astfel de condiții rămâne o problemă distinctă și dificilă, care presupune alte metode pentru determinarea și menținerea poziției.[1] Dronele de dimensiuni mici, capabile să zboare în spații închise formează astfel o categorie aparte, cu cerințe proprii atât la nivel hardware, cât și la nivel software.

1.2 MOTIVAȚIA ȘI SCOPUL PROIECTULUI

Motivația acestei lucrări a pornit din dorința de a înțelege, cât mai profund, cum funcționează un sistem care menține echilibrul și poziția unei drone și de a parcurge personal tot procesul de dezvoltare.

La aceasta se adaugă un obiectiv concret: capturarea de imagini aeriene destinate reconstrucției tridimensionale a unui obiect sau a unei încăperi. Această utilizare impune cerințe diferite de cele ale unei drone obișnuite cu capacități de fotografiere/filmare. Imaginile trebuie realizate în condiții de iluminare cât mai constante între cadrele obținute, altfel programele de reconstrucție interpretează variațiile de luminozitate ca modificări ale scenei reale iar rezultatul se degradează.[2]

Scopul lucrării este, prin urmare, proiectarea, implementarea și testarea unei drone complete, construite de la zero, folosind componente ieftine și larg disponibile.

1.3 SOLUȚIA PROPUȘĂ

Soluția propusă are ca element central o alegere arhitecturală: drona folosește două microcontrolere de mici dimensiuni în loc de unul singur.

Primul se ocupă exclusiv de stabilizarea dronei. Citește permanent date de la senzori și reglează turația celor patru motoare. Al doilea se ocupă de restul sarcinilor: comunicația cu utilizatorul, camera și salvarea imaginilor pe un card de memorie. Această separare a sarcinilor a fost aleasă cu scopul de a preveni posibile perturbări ale sistemului de control cauzate de întârzieri în transmiterea datelor sau operații lente precum salvarea unei fotografii pe cardul de memorie.

Comanda dronei se face printr-o rețea WiFi, creată de unul dintre cele două microcontrolere, folosind o pagină web accesibilă dintr-un browser obișnuit. Acestă

alegere elimină nevoia unei telecomenzi specializate și face drona utilizabilă cu dispozitive pe care oricine le are deja.

În paralel cu programul de pe dronă a fost realizată o aplicație separată, cu rol de depanare și validare folosită pentru ajustarea comportamentului dronei, precum și pentru înregistrarea și analiza ulterioară a zborurilor.

1.4 CONTRIBUȚIA AUTORULUI

Contribuțiile proprii ale acestei lucrări, asumate de autor, sunt următoarele:

- Scrierea logicii de control, integrarea componentelor software necesare senzorilor, parametrizarea și verificarea experimentală a sistemului.
- Arhitectura cu două microcontrolere separate, cu izolarea completă a funcției de zbor față de restul sistemului, reprezintă o abordare diferită de cea a soluțiilor comparabile analizate în capitolul următor, care execută toate funcțiile pe un singur procesor.
- Comanda prin pagină web, fără un echipament radio dedicat, face drona utilizabilă fără achiziția unei telecomenzi specializate.
- Subsistemul de capturare a imaginilor.
- Instrumentul de analiză dezvoltat în paralel a permis identificarea, pe baza datelor înregistrate în timpul zborului, a unor probleme reale de comportament ale sistemului de control, care altfel ar fi rămas invizibile.

Rezultatul nu este un produs finit. O parte dintre parametrii sistemului de control sunt încă valori estimate, iar câteva verificări practice rămân de efectuat. Lucrarea prezintă onest atât ceea ce funcționează, cât și problemele identificate și rămase deschise, considerând că modul în care acestea au fost descoperite și analizate constituie el însuși un rezultat al proiectului.

2 SOLUȚII SIMILARE

Dezvoltarea unui sistem de control de zbor propriu se raportează la un ecosistem matur de soluții existente, care acoperă un spectru larg, de la sisteme de pilotare automată de uz profesional până la platforme de cercetare de dimensiuni reduse și proiecte experimentale bazate pe microcontrolere de uz general.

2.1 SISTEME DE PILOTARE AUTOMATĂ DE UZ PROFESIONAL

Platformele PX4[3] și ArduPilot[4] reprezintă standardul în domeniul sistemelor de pilotare automată open source, fiind utilizate atât în cercetare, cât și în aplicații comerciale. Ambele oferă funcționalități complete de navigație autonomă, planificare de misiune, gestiune a modurilor de zbor și redundanță senzorială, inclusiv suport pentru menținerea poziției în interior pe baza fuziunii dintre un senzor de flux optic și unul de măsurare al distanței.

Aceste sisteme sunt însă proiectate pentru a rula pe platforme hardware dedicate, din familia Pixhawk, construite în jurul unor microcontrolere de clasă superioară și echipate cu unități inerțiale redundante. Volumul codului sursă, cerințele de memorie și complexitatea arhitecturală fac ca aceste soluții să nu fie transferabile pe o placă de dezvoltare de dimensiunea și costul celor utilizate în prezentul proiect.

2.2 FIRMWARE PENTRU DRONE DE MICI DIMENSIUNI

Pentru dronele de mici dimensiuni, categorie în care se încadrează și platforma dezvoltată, soluția dominantă este firmware-ul Betaflight [5]. Acesta este optimizat pentru performanță de zbor manual, cu accent pe latență redusă și pe calitatea răspunsului buclilor de reglare a vitezei unghiulare, și rulează pe controlere de zbor dedicate, construite în jurul microcontrolerelor din familia STM32.

Diferențele față de soluția propusă sunt structurale. Betaflight presupune existența unei legături radio dedicate, cu emițător și receptor specializate, comanda fiind transmisă prin protocoale de radiocomandă consacrate. Subsistemul video, atunci când există, este independent de firmware-ul de zbor: imaginea este transmisă analogic sau digital către ochelarii pilotului, fără a fi procesată sau stocată la bordul aeronavei. Nu există, în consecință, nicio integrare între funcția de zbor și funcția de achiziție de imagini, acestea fiind două sisteme fizic separate care coexistă pe aceeași structură mecanică.

2.3 SOLUȚII BAZATE PE MICROCONTROLERE ESP32

Proiectul ESP-Drone, dezvoltat de Espressif, este cea mai apropiată soluție de tema lucrării: un firmware de zbor pentru microcontrolere din familiile ESP32, ESP32-S2 și ESP32-S3, portat din firmware-ul platformei Crazyflie [6]. Soluția include comandă prin Wi-Fi, dintr-o aplicație mobilă dedicată, iar prin modulul opțional de extensie pentru menținerea poziției utilizează aceeași pereche de senzori: PMW3901 și VL53L1X. Dată fiind această suprapunere, adaptarea acestui proiect la platforma hardware proprie a fost evaluată explicit, ca alternativă la dezvoltarea unui firmware propriu. Analiza a identificat câteva obstacole concrete:

- **Mecanismul bazat pe monitorizarea tensiunii acumulatorului** - ESP-Drone folosește tensiunea acumulatorului atât în lanțul de distribuție a puterii, cât și în mecanismul de supraveghere a stării aeronavei, o funcționalitate care nu era disponibilă sistemului discutat în acest proiect la momentul evaluării.
- **Incompatibilitatea arhitecturală** - ESP-Drone folosește un protocol de comunicație diferit, bazat pe Wi-Fi și care este utilizată împreună cu o aplicație mobilă dedicată. Modulul de telemetrie, tabloul de bord web, protocolul serial dintre cele două plăci, driverul pentru unitatea de măsură inerțială și instrumentul de depanare, dezvoltate în cadrul acestei lucrări, nu au nimic în comun cu acea arhitectură.
- **Coeficienți de reglare ale buclelor PID diferite** - Coeficienții de reglare sunt calibrați pentru structura mecanică diferită, deci ar fi fost necesară oricum o reglare completă, iar dificultățile identificate în timpul testărilor, descrise în capitolul dedicat rezultatelor, provin din parametri specifici acestei construcții, pentru care niciun firmware existent nu oferă valori.

Proiectul a fost folosit, în schimb, ca implementare de referință: modul în care măsurătoarea de flux optic este corectată cu viteza unghiulară măsurată de giroscop, precum și configurarea telemetrului laser, au fost comparate cu soluția adoptată acolo.

3 FUNDAMENTE TEORETICE

În acest capitol sunt prezentate noțiunile teoretice pe care se bazează sistemul realizat, împreună cu uneltele și bibliotecile folosite. Pentru fiecare noțiune este prezentat și modul concret în care aceasta apare implementată.

3.1 PRINCIPIUL DE ZBOR AL UNEI DRONE CU PATRU MOTOARE

La baza funcționării unei drone se află Principiul al 3-lea al Mecanicii lui Newton: pentru fiecare acțiune, există o reacție egală și de sens opus. Mai exact, pe măsură ce elicele dronei împing aerul în jos, ele creează o forță de portanță în sus. În consecință, această forță de portanță permite dronelor să plutească, să urce sau să coboare cu o precizie remarcabilă. Mai mult, prin reglarea vitezei elicelor, dronele își pot controla mișcările și își pot menține stabilitatea în timpul zborului [7].

Pentru a înclina drona într-o parte, motoarele de pe acea parte trebuie să producă mai puțină forță decât cele de pe partea opusă. Diferența dintre forțe creează un moment care rotește drona iar, ca urmare a înclinării, aceasta se deplasează pe axa de translație corespunzătoare. La fel se obține și înclinarea față-spate. Rotația în jurul axei verticale funcționează pe alt principiu. Când o elice se rotește într-un sens, apare un moment de reacție care acționează asupra dronei în sensul opus. Din acest motiv două dintre elice, aflate pe aceeași diagonală, se rotesc în sens orar și două, aflate pe cealaltă diagonală, se rotesc în sens antiorar, astfel încât în zbor normal aceste forțe se anulează reciproc. Dacă accelerăm perechea care se rotește antiorar și le încetinim pe celelalte două, drona se va roti în sens orar.

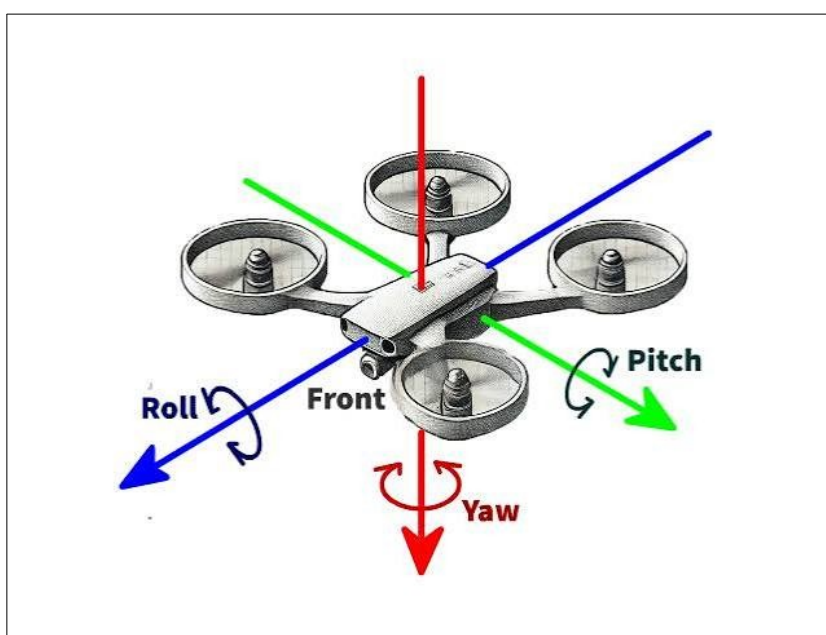


Figura 1 – Principalele mișcări ale unei drone(sursa: [7])

Cele trei comenzi de rotație, ruliu(*roll*), tangaj(*pitch*) și girație(*yaw*) împreună cu comanda de propulsie, trebuie transformate în patru valori individuale, câte una pentru fiecare motor. Această operație este denumită în practică mixare și este implementată astfel:

```
mix[MOTOR_FRONT_LEFT] = roll - pitch - yaw;  
mix[MOTOR_FRONT_RIGHT] = -roll - pitch + yaw;  
mix[MOTOR_REAR_LEFT] = roll + pitch + yaw;  
mix[MOTOR_REAR_RIGHT] = -roll + pitch - yaw;
```

Semnele din fiecare coloană rezultă din poziția motorului și din sensul lui de rotație. Pentru înclinarea laterală, motoarele din stânga primesc semnul plus, iar cele din dreapta semnul minus, astfel încât o comandă pozitivă ridică partea stângă. Pentru înclinarea față-spate, motoarele din spate primesc plus. Pentru rotația pe verticală semnele urmează sensul de rotație al elicei: motoarele care se rotesc antiorar primesc plus.

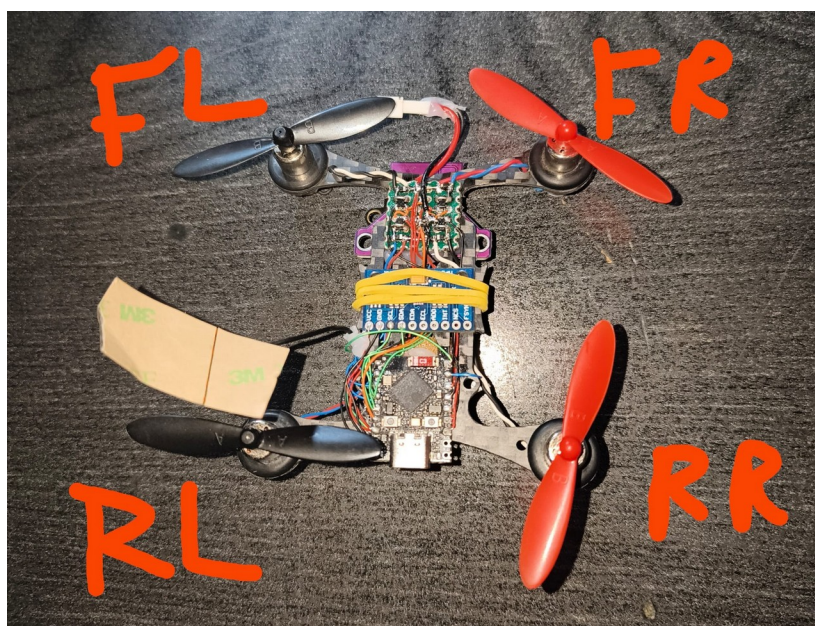


Figura 2 – Poziționarea motoarelor(sursa: proprie)

3.1.1 SUBACTUAREA

Un corp în spațiu are șase grade de libertate: se poate deplasa pe trei axe de translație și pe trei axe de rotație. Drona are însă doar patru comenzi independente, propulsia totală și cele trei momente, *roll*, *pitch* și *yaw*. Se spune că sistemul este subactuat, adică are mai puține comenzi decât grade de libertate.

Consecința este importantă pentru felul în care este organizat programul. Forța produsă de elice este întotdeauna perpendiculară pe corpul dronei, deci, stând dreaptă, drona nu poate produce nicio forță orizontală. Ca să se deplaseze în față, trebuie mai întâi să se încline în față, astfel încât o parte din forța elicelor să acționeze pe orizontală.

Din acest motiv, partea de program care se ocupă de deplasarea orizontală nu comandă direct motoarele, ci calculează un unghi de înclinare, pe care altă parte a programului îl realizează. Această organizare pe niveluri este descrisă în secțiunea 3.3.

3.1.2 PRAGUL MINIM AL MOTOARELOR

Motoarele folosite sunt motoare de curent continuu cu perii. Acestea sunt comandate folosind semnale de tip PWM (*Pulse Width Modulation*). Un astfel de motor comandat de un semnal PWM cu 1% factor de umplere nu se va roti și este posibil ca valoarea factorului de umplere la care motorul începe să se rotească să fie diferită de la motor la motor.

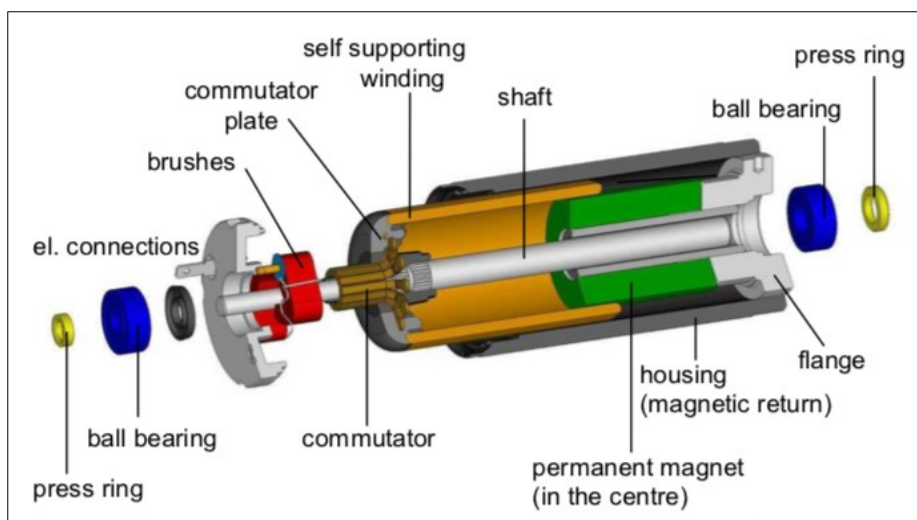


Figura 3 – Motor cu perii de tip *coreless* (sursa: [8])

Funcția de mixare calculează mai întâi doar contribuția de atitudine, fără propulsia totală, ca un set de valori în jurul lui zero. Aceste valori sunt cu semn: o valoare negativă înseamnă că motorul respectiv trebuie să se rotească mai puțin decât media. Comanda finală a fiecărui motor este suma dintre propulsia totală și diferența corespunzătoare din funcția de mixare care poate fi o valoare negativă. Factorul de umplere este limitat la

intervalul 0% -100 % deci o valoare negativă de comandă va rezulta un factor de umplere de 0%, moment în care motorul se oprește, posibil chiar în momentul în care drona ar avea cea mai mare nevoie de corecție. Pentru a evita o astfel de situație, în program este definit un prag minim sub care motoarele nu coboară niciodată cât timp drona este armată.

```
#define MOTOR_IDLE_THRUST 0.06f
```

Această valoare reprezintă un factor de umplere de 6% minim la care toate cele patru motoare se rotesc. Această valoare a fost măsurată experimental, detaliile fiind prezentate în secțiunea de rezultate experimentale.

3.2 REGLAREA AUTOMATĂ PRIN REGULATOR PID

Regulatorul PID este pe departe cel mai comun algoritm de control. Majoritatea buclelor de *feedback* sunt controlate de acest algoritm sau de variații ale acestuia. Este implementată în multe forme diferite, ca regulator independent sau ca un sistem ierarhic distribuit de control al proceselor. Ingineri din toată lumea utilizează acest algoritm în munca lor de zi cu zi [9].

În cazul nostru, pentru ca drona să-și mențină echilibrul nu este suficient să comandăm motoarele o singură dată. Perturbațiile, un curent de aer, o elice ușor dezechilibrată, o greutate distribuită inegal, influențează tipul comenzii necesare motorului. Aici intervine acest tip de regulator în buclă închisă care măsoară orientarea reală, o compară cu cea dorită și corectează comanda în funcție de diferența dintre ele.

Diferența dintre valoarea dorită și cea măsurată se numește eroare:

```
const float error = setpoint - measurement;
```

Regulatorul PID calculează comanda pe baza acestei erori, folosind trei componente: proporțională, integrală și derivativă care la rândul lor utilizează 3 parametri care influențează comportamentul componentei: K_p , K_i , respectiv K_d . În lucrarea citată în acest subcapitol se lucrează cu forma standard a acestor parametri K , T_i și T_d unde T_i și T_d reprezintă constante de timp. Subcapitolele următoare și implementarea de față utilizează forma paralelă a acestor parametri, K_p , K_i și K_d independenți, legătura fiind $T_i = K_p/K_i$ și $T_d = K_d/K_p$ [9].

3.2.1 COMPONENTA PROPORȚIONALĂ

Componenta proporțională generează o comandă direct proporțională cu eroarea curentă:

```
const float p_term = pid->kp * error;
```

Coeficientul de proporționalitate K_p stabilește cum reacționează regulatorul: la o abatere de un grad față de unghiul dorit, comanda produsă este de o mie de ori mai mare pentru un coeficient de o mie decât pentru unul unitar.

Limitarea fundamentală a componentei proporționale este că nu poate anula complet abaterea. Rezultatul este o diferență permanentă între valoarea dorită și cea măsurată, numită eroare staționară (*steady-state error*) [9]. Aceasta apare din cauza modului în care funcționează componenta: comanda de corecție există doar pentru că există eroare. Dacă eroarea ar fi zero, regulatorul nu ar avea ce să corecteze. În prezența unei perturbații permanente, o baterie montată ușor descentrat, un motor care se rotește mai încet decât celelalte la aceeași comandă, bucla va rămărămâner-un punct de echilibru în care eroarea are exact valoarea necesară pentru a genera comanda ce compensează perturbația.

Mărimea acestei erori depinde de coeficientul de proporționalitate K_p : cu cât acesta este mai mare, cu atât abaterea necesară pentru a produce aceeași comandă este mai mică. Eroarea staționară scade, prin urmare, odată cu creșterea coeficientului, dar nu se anulează niciodată, iar dincolo de o anumită valoare bucla va oscila și, în cele din urmă, va deveni instabilă.

Practic, doar cu componenta proporțională, drona ar zbura permanent ușor înclinată, iar înclinarea ar fi cu atât mai mică cu cât reglajul este mai agresiv.

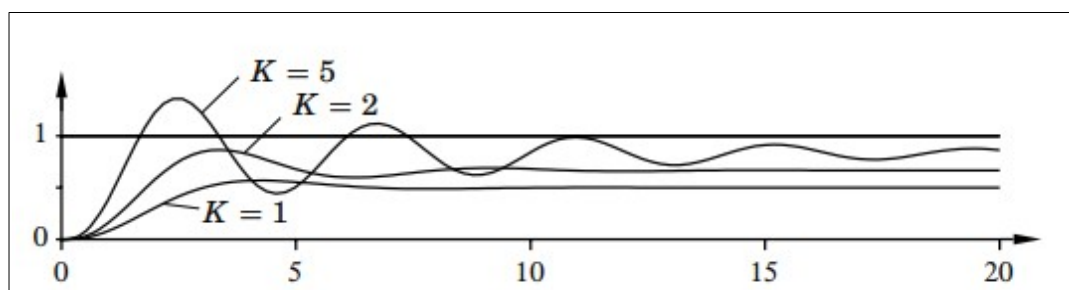


Figura 4 – Simularea unui sistem în buclă închisă folosind doar componenta proporțională. Funcția de transfer a procesului este $G(s)=(s+1)^{-3}$. Graficul arată valoarea dorită (*setpoint*) care este egală cu 1 și ieșirea buclei pentru valori diferite ale K_p . (sursa: [9])

3.2.2 COMPONENTA INTEGRALĂ

Componenta integrală acumulează eroarea în timp și adaugă rezultatul la comandă. Rolul ei este acela de a garanta că, în regim staționar, mărimea reglată ajunge efectiv la valoarea dorită.

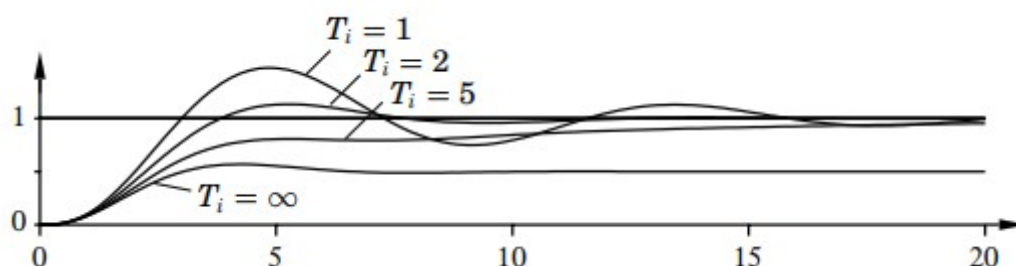
Modul prin care funcționează poate fi înțeles printr-un raționament simplu. Să presupunem că sistemul ar ajunge într-un regim staționar în care eroarea rămâne constantă și diferită de zero. Componenta integrală ar continua atunci să acumuleze, iar comanda ar crește la nesfârșit, ceea ce contrazice ipoteza de regim staționar, în care comanda trebuie să fie constantă. Singura situație în care un regim staționar este posibil

este, prin urmare, aceea în care eroarea este nulă. Prezența componentei integrale elimină astfel eroarea staționară descrisă anterior [9].

Din punct de vedere istoric, această componentă a purtat numele de „resetare automată”: înainte de apariția ei, compensarea unei perturbații permanente se făcea prin ajustarea manuală a unei valori de bază adăugate comenzii, operație care trebuia repetată la fiecare schimbare a condițiilor de funcționare [9].

În cazul dronei, componenta integrală este cea care compensează un dezechilibru mecanic permanent: dacă centrul de greutate nu coincide cu centrul geometric, ea acumulează treptat până când diferența de turație dintre motoare anulează efectul, iar drona zboară orizontal fără ca pilotul să corecteze continuu.

Componenta integrală are însă și dezavantaje. Ea introduce o întârziere suplimentară în răspunsul buclei, reducând stabilitatea, și prezintă fenomenul de



acumulare excesivă descris în secțiunea următoare.

Figura 5 - Simularea unui sistem în buclă închisă folosind componenta proporțională și cea integrală. Funcția de transfer a procesului este $G(s)=(s+1)^{-3}$. Graficul arată valoarea dorită (*setpoint*) care este egală cu 1 și ieșirea buclei pentru $K_p = 1$ și pentru diferite valori ale K_i unde $K_i = K_p/T_i$. (sursa: [9])

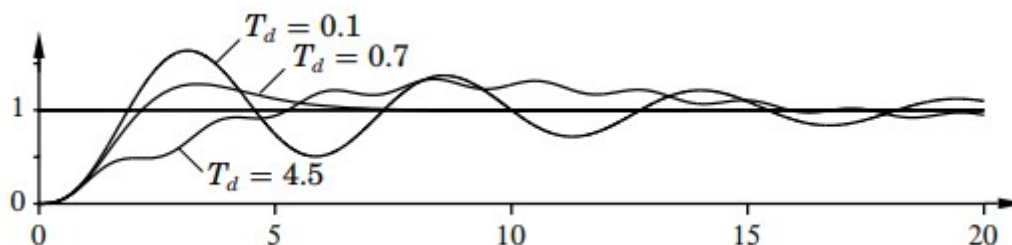
3.2.3 COMPONENTA DERIVATIVĂ

Componenta derivativă este proporțională cu viteza de variație a mărimii reglate. Rolul ei este îmbunătățirea stabilității buclei.

Necesitatea ei constă în faptul că orice sistem fizic răspunde cu întârziere: între momentul în care se modifică turația motoarelor și momentul în care drona își schimbă efectiv orientarea trece un interval de timp. Un regulator care se raportează exclusiv la eroarea prezentă va continua să comande corecție și după ce mișcarea de revenire a început deja, ceea ce produce depășirea valorii dorite și, în consecință, oscilație.

Un regulator proporțional derivativ este, din acest punct de vedere, echivalent cu un regulator proporțional care nu acționează asupra erorii curente, ci asupra erorii anticipate cu T_d secunde în avans, unde $T_d = K_d / K_p$. Constanta T_d reprezintă astfel orizontul de predicție [9].

Practic, dacă drona este înclinată la trei grade, dar se rotește deja rapid înapoi spre orizontală, componenta derivativă reduce corecția, pentru că mișcarea de revenire este



deja în curs și o corecție suplimentară ar produce depășire în sens opus.

Figura 6 - Simularea unui sistem în buclă închisă folosind componentele proporțională, integrală și derivativă. Funcția de transfer a procesului este $G(s)=(s+1)^{-3}$. Graficul arată valoarea dorită (*setpoint*) care este egală cu 1 și ieșirea buclei pentru $K_p = 3$, $K_i = 3$ și diferite valori pentru K_d unde $K_d = K_p \times T_d$ (sursa: [9])

Componenta derivativă ridică însă două probleme practice, ambele cu soluții consacrate. Prima apare la modificarea valorii dorite. În situația în care o comandă se modifică brusc la fel și eroarea se va modifica brusc. Rezultatul este o comandă care nu corespunde niciunei mișcări reale a procesului. Soluția uzuală este calcularea derivatei pe baza mărimii măsurate, nu a erorii. Întrucât diferența dintre cele două este tocmai derivata referinței, eliminarea acestui termen face ca regulatorul să reacționeze exclusiv la evoluția reală a procesului, nu la comenzile operatorului [9].

A doua problemă este amplificarea zgomotului. Caracteristica de frecvență a operatorului de derivare crește proporțional cu frecvența, ceea ce înseamnă că un zgomot de amplitudine mică, dar de frecvență ridicată, inerent oricărei măsurători, cu atât mai mult uneia provenite de la un senzor montat pe o structură care vibrează, produce o contribuție considerabilă în comandă. Din acest motiv componenta derivativă nu se implementează niciodată în formă pură, ci întotdeauna însoțită de o filtrare trece-jos, a cărei frecvență de tăiere stabilește un compromis între atenuarea zgomotului și întârzierea introdusă în răspuns [9].

Modul concret în care aceste două soluții au fost implementate, împreună cu valorile alese pentru fiecare buclă, este prezentat în capitolul următor.

3.2.4 SATURAREA ȘI ACUMULAREA EXCESIVĂ

Orice actuator are limitări. În cazul de față, viteza la care se pot roti motoarele utilizate este limitată. În cazul în care regulatorul cere mai mult decât pot motoarele livra, sistemul intră în saturație iar bucla de control se deschide: oricât ar crește comanda calculată, motoarele nu vor mai răspunde suplimentar.

Problema apare la componenta integrală. Aceasta nu are cum să știe că sistemul este saturat și continuă să adune eroarea, care nu scade. Valoarea acumulată poate crește foarte mult, iar când situația se normalizează ea trebuie mai întâi consumată, timp în care comanda rămâne la maxim deși nu mai este nevoie. Fenomenul poartă numele de *integrator windup*.

O soluție care previne acest fenomen constă în integrarea condiționată. Înainte de a accepta pasul de integrare, programul va calcula ce comandă ar rezulta și numai dacă eroarea curentă ar împinge comanda și mai departe în saturație aceasta nu se mai acumulează. Modul concret de implementare va fi prezentat în capitoul următor.

3.3 STRUCTURA DE REGLARE ÎN CASCADĂ

Sistemul nu folosește un singur regulator, ci mai multe, organizate pe trei niveluri. Fiecare nivel stabilește valoarea dorită pentru nivelul de sub el: nivelul exterior calculează, din altitudine și din viteza de deplasare, comanda de propulsie și unghiurile de înclinare dorite; nivelul intermediar transformă unghiul dorit în viteză unghiulară dorită; nivelul interior transformă viteza unghiulară dorită în comenzi către motoare.

Motivul acestei organizări a fost explicat în secțiunea privind subactuarea: drona nu poate comanda direct deplasarea, ci doar înclinarea.

Frecvențele de execuție diferă pentru că mărimile se schimbă cu viteze diferite. Viteza unghiulară se poate modifica foarte repede și este măsurată de giroscop la fiecare ciclu, deci bucla interioară rulează cel mai des. Unghiul se schimbă mai lent. Altitudinea și viteza de deplasare se schimbă cel mai lent și, în plus, senzorii care le măsoară furnizează date mai rar: telemetrul laser în jur de 33 de măsurători pe secundă.

În program, cele trei bucle nu rulează ca sarcini separate, ci sunt derivate dintr-un singur ciclu de 1000 Hz, prin numărare:

```
#define INNER_LOOP_HZ      1000
#define MID_LOOP_DIVIDER   4      // 1000 / 4 = 250 Hz
#define OUTER_LOOP_DIVIDER 20     // 1000 / 20 = 50 Hz
```

Avantajul este că cele trei bucle nu se pot defaza una față de alta. Dacă ar fi sarcini separate, întârzierea dintre calculul unei valori dorite și folosirea ei ar varia de la un ciclu la altul, ceea ce ar face comportamentul mai greu de analizat.

Regula uzuală pentru astfel de structuri este ca fiecare buclă interioară să ruleze de cel puțin cinci ori mai des decât cea exterioară, astfel încât, din punctul de vedere al buclei exterioare, cea interioară să pară instantanee [10]. În implementarea realizată raportul dintre nivelul exterior și cel intermediar este 5, deci respectă recomandarea; între nivelul intermediar și cel interior este 4, ușor sub valoarea recomandată. O consecință practică a acestei structuri este ordinea în care trebuie făcută reglarea: dinspre interior spre exterior. Fiecare buclă exterioară presupune că bucla de sub ea funcționează deja corect, deci nu are sens să fie reglată înaintea acesteia.

3.4 ESTIMAREA ORIENTĂRII PRIN FUZIUNEA SENZORILOR

Pentru a corecta orientarea, sistemul trebuie mai întâi să o cunoască. Drona nu dispune de un senzor care să indice direct unghiul de înclinare, ci de un accelerometru și un giroscop, iar unghiul trebuie calculat din datele furnizate de acestea.

3.4.1 ACCELEROMETRUL

Accelerometrul măsoară accelerația care acționează asupra lui. Este un dispozitiv electromecanic care detectează fie forțe statice, fie forțe dinamice de accelerație. Forțele statice includ gravitația, în timp ce forțele dinamice pot include vibrații și mișcare [11].

Această proprietate este utilă. Când drona stă nemișcată sau se deplasează uniform, singurul lucru pe care îl măsoară accelerometrul este gravitația. Dacă drona este perfect orizontală, toată accelerația apare pe axa verticală; dacă este înclinată, o parte din ea se regăsește pe celelalte axe, iar din raportul dintre componente se poate calcula unghiul [12]:

```
*roll_angle = (atan2(acc_y_g, acc_z_g) * (180.0 / M_PI)) - roll_offset;  
*pitch_angle = (atan2(-acc_x_g, sqrt(acc_y_g * acc_y_g + acc_z_g * acc_z_g))  
                * (180.0 / M_PI)) - pitch_offset;
```

Valorile scăzute la final sunt corecții măsurate la pornire, cu drona așezată orizontal, pentru a compensa faptul că senzorul nu este montat perfect drept pe placă.

Accelerometrul are însă două probleme. Prima este că nu poate distinge între înclinare și accelerație: dacă drona accelerează lateral fără să se încline, senzorul indică practic același lucru ca în cazul unei înclinări. A doua este că vibrațiile produse de elice îl afectează puternic.

3.4.2 GIROSCOPUL

Giroscopul măsoară o altă mărime decât accelerometrul: viteza de rotație, exprimată în grade pe secundă. Nu are nicio legătură cu gravitația și nu este afectat de accelerațiile liniare.

Obținerea unghiului se face prin integrare: la fiecare ciclu, unghiul crește cu viteza măsurată înmulțită cu timpul scurs de la ciclul anterior. Aici apare însă problema fundamentală a acestui senzor, integrarea nu acumulează doar semnalul util, ci și erorile de măsurare.

Două tipuri de eroare contează, iar ele se acumulează diferit. Primul este *offsetul* constant: senzorul indică o valoare mică, diferită de zero, chiar și atunci când stă perfect nemișcat. Integrat, acest *offset* produce o eroare de unghi care crește liniar cu timpul. O abatere de doar 0,5 grade pe secundă devine, după un minut de zbor, o eroare de 30 de grade. Al doilea este zgomotul aleatoriu, provenit din agitația termică a structurii mecanice a senzorului. Integrat, acesta produce o deplasare aleatoare a unghiului estimat, a cărei abatere crește proporțional cu rădăcina pătrată a timpului. Efectul este mai lent decât cel al *offsetului*, dar nu poate fi eliminat prin calibrare, fiind aleatoriu prin natura lui [13].

La acestea se adaugă faptul că *offsetul* însuși nu este strict constant, ci variază lent cu temperatura și în timp, ceea ce produce tot o eroare cu creștere liniară. Contramăsura uzuală este determinarea *offsetului* la pornire, cu senzorul nemișcat: se măsoară valoarea indicată timp de câteva sute de eșantioane, se calculează media, iar aceasta se scade din toate citirile ulterioare. Procedura elimină componenta constantă existentă la momentul pornirii, dar nu rezolvă variația ulterioară cu temperatura și nu are niciun efect asupra zgomotului aleatoriu [13].

Concluzia practică este că un giroscop nu poate furniza singur orientarea absolută a unei aeronave pe durata unui zbor. Oricât de bine ar fi calibrat, estimarea obținută prin integrare se îndepărtează treptat de realitate și nu se poate corecta singură, întrucât nu dispune de nicio referință externă față de care să se compare.

Astfel cele două surse sunt complementare: accelerometrul este corect pe termen lung dar zgomotos, giroscopul este precis pe termen scurt dar derivează. Modul în care sunt combinate este prezentat în capitolul 4.

3.5 MĂSURAREA ALTITUDINII ȘI A VITEZEI DE DEPLASARE

3.5.1 TELEMETRUL LASER

Pentru măsurarea altitudinii este folosit un senzor ToF (Time of Flight) VL53L1X [14], orientat în jos. Acesta emite un impuls luminos infraroșu și măsoară timpul până când lumina se întoarce, reflectată de podea. Cunoșcând viteza luminii, distanța rezultă direct din acest timp.

Măsurătoarea este considerată validă doar în anumite condiții, verificate în program: distanța trebuie să fie cuprinsă între 3 cm și 1,30 m, iar drona nu trebuie să fie înclinată prea mult, pentru că altfel fasciculul nu mai lovește podeaua de sub senzor, ci o zonă laterală, aflată la altă distanță.

3.5.2 SENZORUL DE FLUX OPTIC

Pentru măsurarea vitezei de deplasare pe orizontală este folosit un senzor PMW3901 [15], orientat în jos. El funcționează pe același principiu ca senzorul optic dintr-un mouse de calculator: fotografiază podeaua foarte des și compară imaginile consecutive pentru a determina cu cât s-a deplasat imaginea între ele.

Un aspect esențial este că senzorul nu măsoară o distanță, ci o deplasare unghiulară. El indică cu cât s-a mișcat imaginea în câmpul lui vizual, nu câți centimetri s-a deplasat drona. Aceeași deplasare unghiulară înseamnă câțiva centimetri dacă drona este aproape de sol și considerabil mai mult dacă este sus.

Din acest motiv, pentru a obține viteza în metri pe secundă este obligatoriu să fie cunoscută și altitudinea [16]:

```
const float flow_x_rad = flow->delta_x / FLOW_COUNTS_PER_RAD;  
...  
const float raw_velocity_x = (translation_x_rad / dt) * state.altitude;
```

Aceasta este și explicația pentru care cei doi senzori nu sunt interschimbabili și niciunul nu poate fi eliminat: fără telemetru nu există altitudine, iar fără altitudine măsurătoarea de flux optic nu poate fi convertită în viteză.

Înainte de calcularea vitezei, programul mai face o corecție. Dacă drona se înclină fără să se deplaseze, imaginea de sub ea se mișcă oricum, iar senzorul ar raporta o deplasare care nu a avut loc. Din măsurătoarea de flux se scade, de aceea, componenta datorată rotației, măsurată de giroscop.

3.6 EXECUȚIA PROGRAMULUI ÎN TIMP REAL

Toate calculele descrise anterior au sens doar dacă sunt executate precis din punctul de vedere la ordini efectuării sarcinilor, priorității sarcinilor și a momentului în care se execută. Un regulator care ar trebui să ruleze la fiecare milisecundă, dar rulează uneori la două milisecunde, calculează greșit componentele integrală și derivativă, pentru că ambele depind de timpul scurs.

Pentru gestionarea acestui aspect, programul folosește sistemul de operare în timp real FreeRTOS, inclus în *framework*-ul ESP-IDF [17]. Acesta permite împărțirea programului în sarcini independente, fiecare cu propria prioritate, și garantează că o sarcină cu prioritate mare o întrerupe pe una cu prioritate mai mică.

Microcontrolerul ESP32-S3 dispune de două nuclee de procesare identice denumite Core 0, deseori referit ca și PRO_CPU, și Core1, deseori referit ca și APP_CPU. Aceste denumiri există cu scopul de a reflecta cum utilizează o aplicație ESP-IDF tipică cele două nuclee. Sarcinile responsabile cu utilizarea protocoalelor de comunicație precum WiFi sau Bluetooth rulează pe Core 0 iar restul sarcinilor rulează pe Core1 cu toate că o sarcină poate fi fixată pe oricare dintre cele două nuclee [17].

În implementarea realizată, bucla de control rulează pe nucleul 1, cu cea mai mare prioritate, iar comunicația cu modulul de telemetrie pe nucleul 0. Astfel, o întârziere apărută în comunicație nu poate întârzia bucla de control.

Din același motiv, în bucla de control nu se execută operații a căror durată nu poate fi garantată: fără citiri de pe rețea, fără așteptare după alte sarcini.

Atunci când două sarcini aflate pe nuclee diferite folosesc aceleași date, de exemplu comenzile primite de la pilot, scrise de sarcina de comunicație și citite de bucla de control, accesul este protejat, astfel încât bucla de control să nu poată citi jumătate din comanda veche și jumătate din cea nouă.

3.7 MEDIUL DE DEZVOLTARE ȘI BIBLIOTECILE UTILIZATE

Firmware-ul ambelor plăci este dezvoltat folosind ESP-IDF (Espressif IoT Development Framework), mediul oficial de dezvoltare pus la dispoziție de Espressif pentru microcontrolerele din familia ESP32. Acesta oferă un lanț de instrumente, API-uri, componente și fluxuri de lucru pentru dezvoltarea de aplicații pentru ESP32-S3 utilizând sistemele de operare Windows, Linux și macOS [18].

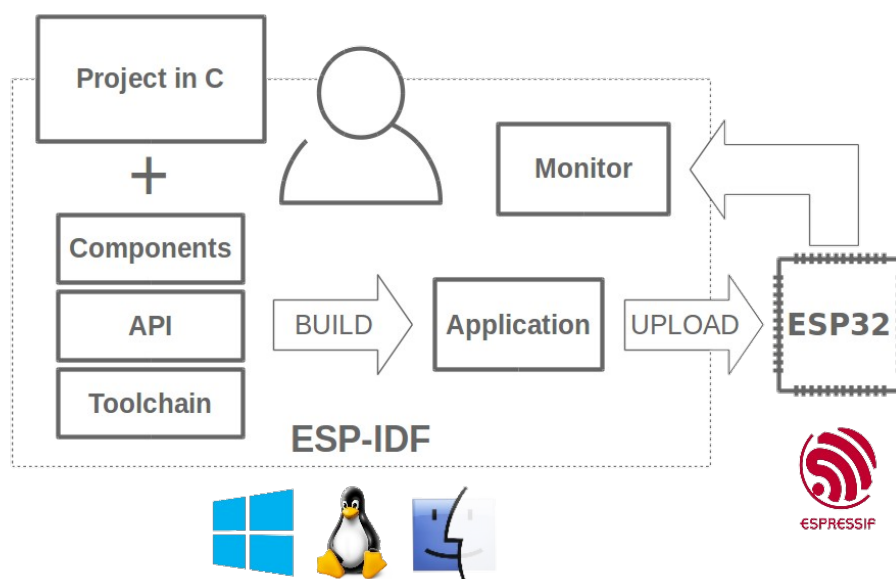


Figura 7 – Funcționarea mediul de lucru ESP-IDF (sursa: [18])

Un proiect ce utilizează acest mediu de dezvoltare este organizat în componente. O componentă este un director care conține codul sursă, fișierele antet publice și un fișier de configurare în care se declară de ce alte componente depinde. Compilarea este gestionată prin CMake, iar sistemul determină automat ordinea de compilare pe baza dependențelor declarate.

Această organizare a fost utilă pentru proiect: fiecare funcție majoră a devenit o componentă separată, driverul senzorului inerțial, estimatorul de orientare, estimatorul de navigație, regulatorul PID, bucla de control, driverul de motoare, monitorizarea acumulatorului, cu limite clare între ele. O componentă poate folosi altă componentă doar dacă a declarat explicit dependența, ceea ce împiedică apariția legăturilor accidentale între părți care ar trebui să rămână independente.

Pe lângă componentele scrise în cadrul proiectului au fost folosite și biblioteci externe. Pentru telemetrul laser este utilizat driverul oficial pus la dispoziție de producător, integrat printr-un strat de adaptare scris în cadrul proiectului, care implementează funcțiile de comunicație pe magistrala I2C. Pentru senzorul de flux optic, secvența de inițializare a registrelor este preluată din driverul cu sursă deschisă publicat de Bitcraze, cu atribuirea și licența păstrate în codul sursă. Modulul de cameră folosește driverul esp32-camera, pus la dispoziție de Espressif. La acestea se adaugă driverele incluse în ESP-IDF pentru perifericele microcontrolerului: magistralele I2C și SPI, interfața serială, generatorul de PWM folosit pentru comanda motoarelor și convertorul analog-digital.

Cele două plăci constituie două proiecte ESP-IDF separate, care partajează o singură componentă comună, cea care definește structura mesajelor schimbate între ele.

Componenta este inclusă în ambele proiecte, astfel încât o modificare a formatului mesajelor se aplică automat de ambele părți și nu pot apărea nepotriviri între ceea ce trimite o placă și ceea ce așteaptă cealaltă.

4 ELABORAREA PROIECTULUI

Capitolul anterior a prezentat noțiunile teoretice pe care se sprijină sistemul. Capitolul de față descrie modul în care acestea au fost transpuse într-un program care rulează pe hardware real: ce s-a cerut de la sistem, cum a fost împărțit, ce decizii de proiectare au fost luate și pe ce argumente, cum arată implementarea și cum a fost verificată.

Pentru fiecare valoare numerică aleasă se indică motivul care a dus la ea. Acolo unde o valoare nu a fost determinată experimental, ci estimată, acest lucru este spus explicit: o parte dintre parametrii sistemului sunt încă valori de plecare. Referințele către codul sursă indică fișierul și linia, astfel încât fiecare afirmație să poată fi verificată direct.

4.1 SPECIFICAȚIA SISTEMULUI

Specificația a fost formulată înaintea implementării și a rămas neschimbată pe parcursul proiectului. Este împărțită în cerințe funcționale, adică ce trebuie să facă sistemul, și cerințe nefuncționale, adică în ce condiții trebuie să o facă, și constrângeri impuse de hardware-ul ales.

4.1.1 CERINȚE FUNCȚIONALE

Sistemul trebuie să îndeplinească următoarele funcții:

- să stabilizeze automat orientarea aeronavei, astfel încât pilotul să comande o înclinare dorită, nu turația fiecărui motor în parte;
- să permită menținerea automată a altitudinii și a poziției pe orizontală, folosind exclusiv senzorii aflați la bord, fără nicio referință externă de tip GPS;
- să poată fi comandat fără telecomandă dedicată, de pe un dispozitiv pe care pilotul îl are deja;
- să captureze imagini pe un card de memorie;
- să permită modificarea coeficienților de reglare în timpul funcționării, fără reprogramarea plăcii;

4.1.2 CERINȚE NEFUNCȚIONALE

Cea mai restrictivă cerință a sistemului constă în frecvența la care trebuie să lucreze bucla interioară, cea care reglează viteza unghiulară a dronei. Aceasta trebuie să ruleze la 1000 Hz, deci are un termen limită de o milisecundă pe iterație în care trebuie să aibă loc citirea unității inerțiale pe magistrala I2C, actualizarea estimatorului de orientare, actualizarea reglatoarelor de viteză unghiulară, operația de mixare și comandarea celor patru motoare prin PWM.

Din această cerință derivă o regulă de proiectare care influențează întregul program. Bucla interioară nu așteaptă niciodată. Fiecare citire între sarcinile din sarcina principală a controlerului de zbor folosește un timp de așteptare egal cu zero. În cazul în

care citirea eșuează se continuă cu datele obținute anterior. Această alegere a fost făcută pe principiul că este mai bine să fie folosite date din iterația anterioară decât desincronizarea sistemului cauzat de o citire lentă sau nereușită.

```
bool sensor_task_get_nav_state(nav_state_t *out) {
    if (out == NULL || nav_state_mutex == NULL) {
        return false;
    }

    // Zero timeout: the 1 kHz control loop must never block waiting on the 100
    Hz sensor task.

    // A missed copy just means fc_task reuses last iteration's nav state,
    which is at most

    // 1 ms stale and completely harmless for a 50 Hz outer loop.
    if (xSemaphoreTake(nav_state_mutex, 0) != pdTRUE) {
        return false;
    }

    *out = shared_nav_state;
    xSemaphoreGive(nav_state_mutex);

    return true;
}
```

flight_controller/main/main.c, liniile 15–18

A doua cerință nefuncțională este de siguranță anume: nicio cale de execuție care nu autorizează explicit motoarele nu are voie să le lase în mișcare. Modul în care este garantată această proprietate este descris în secțiunea 4.4.6.

4.1.3 CONSTRÂNGERI IMPUSE DE HARDWARE

Alegerile de hardware, prezentate în subcapitolul 1.3, au consecințe directe asupra structurii programului. Ele nu sunt detalii de implementare, ci constrângeri de proiectare:

Constrângere	Consecință asupra proiectării
Motoare de curent continuu cu perii, comandate prin tranzistoare	Comanda este un factor de umplere PWM, nu un protocol digital de tip DShot. Nu există confirmare de la motor și nu există măsurare a turației.
Absența unui magnetometru	Direcția absolută nu este observabilă. Girația nu poate avea buclă de unghi, ci doar de viteză unghiulară.
Absența unui receptor GPS	Poziția pe orizontală se estimează exclusiv din flux optic și altitudine, ambele valabile doar aproape de sol.
O singură magistrală I2C, partajată între unitatea inerțială și telemetrul laser	Ordinea de inițializare devine o constrângere pentru a menține funcționarea corectă a programului (secțiunea 4.4.9).
Card SD cu timp de scriere nedeterminist	Capturarea imaginilor nu poate rula pe aceeași cale de execuție cu comanda aeronavei.
Alimentare de la un acumulator 1S de capacitate mică	Puterea de emisie radio și numărul de sarcini active devin decizii cu impact asupra autonomiei.

Tabelul 1 – Constrângerile hardware și consecințele lor asupra structurii programului

4.1.4 DELIMITAREA FUNCȚIONALITĂȚILOR

Pentru claritate, este util de precizat și ce nu face sistemul. Nu există planificare autonomă de traiectorie: drona execută comenzile pilotului, iar modurile automate se limitează la menținerea altitudinii și a poziției curente. Nu există evitarea obstacolelor. Nu se realizează nicio prelucrare a imaginilor la bord, iar reconstrucția tridimensională propriu-zisă nu face obiectul acestei lucrări; sistemul produce setul de imagini, nu scena reconstruită din el.

4.2 ARHITECTURA SOFTWARE A SISTEMULUI

4.2.1 ÎMPĂRȚIREA SISTEMULUI PE DOUĂ MICROCONTROLERE

Decizia arhitecturală centrală, menționată în subcapitolul 1.3, este separarea sistemului pe două microcontrolere distincte, în locul unuia singur.

Separarea părții critice a sistemului de cea mai puțin critică, anume modulul de telemetrie, cu scopul de a elimina șansele ca acesta să fie perturbat reprezintă motivul principal pentru care a fost luată această decizie arhitecturală.

Bucula de control are un termen limită de o milisecundă. Scrierea unei fotografii pe un card SD poate bloca sute de milisecunde atunci când cardul își execută operațiile interne, iar durata acestei blocări nu poate fi nici precisă, nici limitată de program. Pe un singur microcontroler, cele două activități ar concura pentru aceleași resurse. Fiind separate pe plăci diferite, nicio întârziere din partea de imagini nu poate ajunge la bucla de control, indiferent cât durează.

Dezavantajul cauzat de această separare este o legătură serială între plăci, care trebuie proiectată, implementată și verificată. Ea face obiectul subcapitolului 4.3.

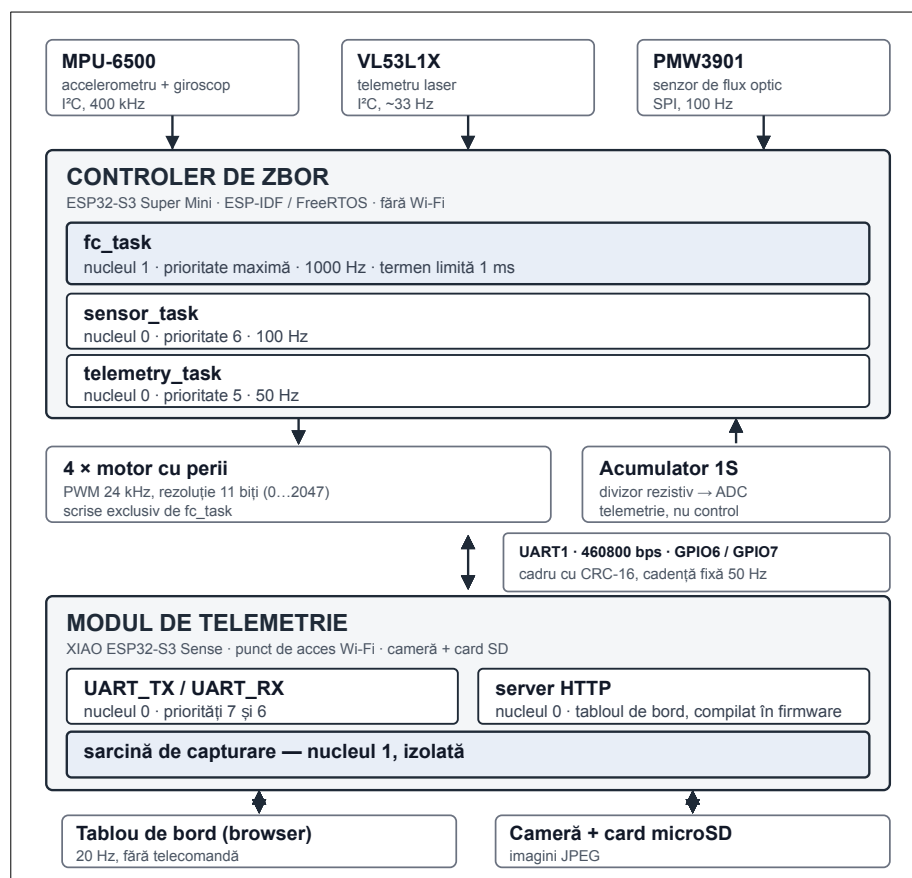


Figura 8 – Arhitectura sistemului pe două plăci, cu sarcinile, nucleele și legăturile dintre ele(sursa: proprie, generat cu IA)

4.2.2 ORGANIZAREA PE COMPONENTE

Fiecare funcție majoră a firmware-ului este o componentă ESP-IDF separată, cu limite explicite. Organizarea nu este decorativă: sistemul de compilare impune ca o componentă să poată folosi altă componentă doar dacă a declarat explicit dependența, ceea ce împiedică apariția legăturilor accidentale între părți care ar trebui să rămână independente. Controlerul de zbor este format din opt componente proprii și două drivere externe:

Componentă	Rol
imu_driver	Comunicația I2C cu unitatea inerțială, conversia datelor brute în unități fizice, calibrarea la pornire
attitude_estimator	Filtrul complementar: din accelerație și viteze unghiulare rezultă unghiurile de orientare
nav_estimator	Din distanță și flux optic rezultă altitudinea și viteza de deplasare pe orizontală
pid_controller	Legea de reglare propriu-zisă, fără nicio stare globală
pid_registry	Tabelul de coeficienți, coada de depanare și generatorul de semnal de test
flight_control	Cascada de reglare, mașina de stări de armare, mixerul
motor_driver	Cele patru canale PWM și conversia din propulsie normalizată în factor de umplere
battery_monitor	Tensiunea acumulatorului, măsurată printr-un divizor rezistiv și convertorul analog-digital
vl53l1x_driver	Driverul oficial al producătorului, plus stratul de adaptare I2C scris în cadrul proiectului
pmw3901_driver	Driver propriu, cu secvența de inițializare a registrelor preluată de la Bitcraze

Tabelul 2 – Componentele firmware-ului de pe controlerul de zbor

Separarea are un efect concret asupra implementării regulatorului. Interfața componentei `pid_controller` nu conține nicio variabilă globală: fiecare funcție primește ca prim argument un pointer către structura care ține starea, iar structura aparține apelantului. Componenta oferă doar algoritmul, nu și instanțele lui.

Interfața componentei - fiecare funcție primește un pointer către structura apelantului:

```
typedef struct {  
float kp, ki, kd;  
float integrator;  
float integrator_limit;  
float prev_measurement;  
float d_filtered;  
float d_alpha;  
float out_min, out_max;  
... } pid_controller_t;  
  
esp_err_t pid_init(pid_controller_t *pid, float kp, float ki, float kd, ...);  
float      pid_update(pid_controller_t *pid, float setpoint, float measurement,  
float dt);  
void      pid_reset(pid_controller_t *pid);  
void      pid_set_gains(pid_controller_t *pid, float kp, float ki, float kd);
```

flight_controller/components/pid_controller/include/pid_controller.h:38–112

Apelantul - cele opt instanțe sunt declarate și deținute de bucla de control, nu de componenta PID:

```
static pid_controller_t pid_loops[TELEMETRY_LOOP_COUNT];  
...  
static float flight_control_run_pid(pid_loop_id_t id,  
float setpoint, float measurement,  
float dt, float now_s) {  
pid_controller_t *pid = &pid_loops[pid_loop_to_telemetry[id]];  
...  
const float output = pid_update(pid, injected_setpoint, measurement, dt);
```

flight_controller/components/flight_control/flight_control.c:185, 279–285

Consecința se vede în bucla de control, unde cele opt regulatoare sunt un simplu vector: fiecare element este o instanță completă și independentă, cu proprii coeficienți, propriul integrator și propriul istoric al derivatei. Nu există nimic partajat între ele, deci nu există nici ceva de protejat prin sincronizare, chiar dacă buclele rulează la trei cadențe diferite în cadrul aceleiași sarcini. Aceeași implementare acoperă atât o buclă de viteză

unghiulară la 1000 Hz, cât și una de altitudine la 50 Hz, fără nicio ramificație care să distingă între ele.

Modulul de telemetrie este organizat după același principiu, cu șase componente proprii:

- `control_state` pentru sinteza comenzilor
- `uart_link` pentru legătura serială
- `wifi_ap` pentru punctul de acces
- `http_server` pentru tabloul de bord
- `gs_link` pentru puntea către stația de sol
- `camera_sd` pentru capturarea imaginilor

4.2.3 REPARTIZAREA SARCINILOR PE NUCLEE ȘI PRIORITĂȚI

Ambele microcontrolere au două nuclee, iar sistemul de operare în timp real permite fixarea fiecărei sarcini pe un nucleu anume, prin funcția `xTaskCreatePinnedToCore()` [18]. Repartizarea aleasă este următoarea:

Placă	Sarcină	Nucleu	Prioritate	Frecvență	Fișier și linie
Controler de zbor	<code>fc_task</code>	1	maximă	1000 Hz	<code>main.c:542</code>
Controler de zbor	<code>sensor_task</code>	0	6	100 Hz	<code>sensor_task.c:45</code>
Controler de zbor	<code>telemetry_task</code>	0	5	50 Hz	<code>main.c:576</code>
Modul de telemetrie	<code>UART_TX</code>	0	7	50 Hz	<code>uart_link.c:50</code>
Modul de telemetrie	<code>UART_RX</code>	0	6	continuu	<code>uart_link.c:51</code>
Modul de telemetrie	capturare imagini	1	implicită	aperiodic	<code>main.c,</code> <code>CORE_CAMERA</code>

Tabelul 3 – Repartizarea sarcinilor pe nuclee, priorități și cadențe

Ultima sarcină este aperiodică: nu rulează la un interval fix, ci rămâne blocată până când apare o cerere de fotografiere.

Atribuirea priorităților a fost gândită în funcție de perioada la care lucrează sarcina respectivă. Sarcina cu perioada cea mai scurtă, care se execută cel mai des, primește prioritatea cea mai mare. Bucla de control are perioada de 1 ms și primește prioritatea maximă disponibilă; sarcina de senzori are 10 ms și prioritatea 6; sarcina de telemetrie are 20 ms și prioritatea 5.

Pe modulul de telemetrie, sarcina de transmisie este plasată deasupra celei de recepție dintr-un motiv concret: ratarea unui slot de transmisie riscă să declanșeze watchdog-ul de 300 ms al controlerului și să dezarmeze drona în zbor, în timp ce un cadru de stare recepționat cu întârziere nu va influența critic sistemul. Cele două consecințe nu sunt comparabile ca gravitate, deci nici prioritățile nu trebuie să fie egale.

Separarea sarcinilor pe nuclee folosită la controlerul de zbor nu este motivată aici de traficul radio, ci de izolarea buclei de control față de sarcinile proprii ale aplicației care execută operații de intrare-ieșire cu durată nedeterministă: sarcina de senzori face citiri I2C care pot dura milisecunde, iar cea de telemetrie scrie pe interfața serială. Ambele rulează pe nucleul 0, iar bucla de control ocupă singură nucleul 1.

Pe modulul de telemetrie situația este cea uzuală pentru o aplicație ESP-IDF: Wi-Fi și stiva de rețea rulează oricum pe nucleul 0, împreună cu serverul HTTP și cele două sarcini seriale, astfel încât întreaga cale de comandă este planificată de un singur nucleu, eliminând eventuale lăntențe. Sarcina de capturare a imaginilor este singura de pe nucleul 1, tocmai pentru că este singura care blochează pentru perioade lungi și impredictibile.

4.2.4 SCHIMBUL DE DATE ÎNTRE SARCINI

Sarcinile nu comunică prin apeluri directe, ci prin structuri partajate, protejate diferit în funcție de natura datelor. Trei mecanisme sunt folosite, iar alegerea fiecăruia are o motivație distinctă.

Spinlock pentru comanda pilotului. Comanda primită de la modulul de telemetrie este scrisă de o sarcină de pe nucleul 0 și citită de bucla de control de pe nucleul 1. Copierea structurii nu este atomică, deci fără protecție bucla ar putea citi jumătate din comanda veche și jumătate din cea nouă. Secțiunea critică este foarte scurtă, doar o copie de structură, ceea ce justifică folosirea unui spinlock în locul unui mutex:

```
taskENTER_CRITICAL(&control_input_spinlock);
control = control_input;
input_age_us = now_us - last_control_input_us;
taskEXIT_CRITICAL(&control_input_spinlock);
```

flight_control.c:832–835

Mutex asimetric pentru starea de navigație. Sarcina de senzori publică rezultatul la 100 Hz, iar bucla de control îl consumă la 1000 Hz. Cele două capete folosesc timeout-uri diferite, iar asimetria este chiar proiectarea: scriitorul acceptă să aștepte până la 2 ms din bugetul lui de 10 ms și renunță la o publicare dacă mutexul este ocupat, în timp ce cititorul folosește timeout zero și păstrează copia din iterația anterioară. Partea de 1 kHz

nu așteaptă niciodată, iar o valoare veche cu o milisecundă este complet inofensivă pentru o mărime care se actualizează oricum de zece ori mai rar.

Semafor pentru ordinea de pornire. Un semafor binar sincronizează un eveniment care se produce o singură dată: momentul în care magistrala I2C a fost creată și senzorii de navigație pot fi porniți fără să concureze pentru ea. Detaliile sunt în secțiunea 4.4.9.

4.3 PROTOCOLUL DE COMUNICAȚIE ÎNTRE PLĂCI

4.3.1 STRUCTURA CADRULUI

Legătura dintre cele două plăci este serială asincronă, la 460800 biți pe secundă, pe interfața UART1, cu transmisia pe GPIO6 și recepția pe GPIO7 (main.c:526–531). Viteza a fost aleasă suficient de mare încât un cadru complet să ocupe o fracțiune neglijabilă din perioada de 20 ms: la 460800 bps, un cadru de 69 de octeți se transmite în aproximativ 1,5 ms.

Structura unui cadru este prezentată în tabelul următor:

0xAA 1 octet	Tip 1 octet	Lungime 1 octet	Secv. 2 octeți	Date utile 0...64 octeți	CRC-16 2 octeți	0xBB 1 octet
-----------------	----------------	--------------------	-------------------	-----------------------------	--------------------	-----------------

Cei opt octeți de încadrare sunt necesari pentru că o linie serială nu oferă nici delimitare de cadru, nici detecție de erori. Este util de comparat cu legătura peste Wi-Fi descrisă în secțiunea 4.5.5, unde exact din motivul opus se folosește un format mai simplu: UDP oferă deja ambele proprietăți, deci repetarea lor ar fi o risipă.

Pentru detecția erorilor se folosește CRC-16/CCITT-FALSE, cu polinomul generator 0x1021 și valoarea inițială 0xFFFF, calculat bit cu bit (telemetry_uart.c:44–58). Este un algoritm standardizat și larg folosit în protocoale seriale [19]. Implementarea cu operații pe biți are dezavantajul ca este mai lentă decât folosirea unei tabele de căutare dar pentru un cadru de maxim 69 de octeți diferența este neglijabilă.

În calculul CRC-ului intră toate câmpurile cadrului înafară de cele care marchează începutul și sfârșitul cadrului și câmpul care stochează valoarea CRC calculată. Consecința este că octetul de start 0xAA nu decide validitatea unui cadru; el este doar o indicație despre unde merită căutat începutul unui cadru. Un octet 0xAA care apare din întâmplare în interiorul datelor utile va fi încercat ca început de cadru și respins câteva poziții mai târziu, la verificarea lungimii, a octetului de sfârșit sau a sumei de control. Tocmai de aceea nu există niciun mecanism de marcare a datelor: suma de control este cea care decide, iar un mecanism suplimentar ar adăuga complexitate fără să adauge siguranță (telemetry_uart.c:177–180).

4.3.2 TIPURILE DE MESAJE

Protocolul definește unsprezece tipuri de mesaje (telemetry_uart.h:35–51), grupate pe direcții. Notățiile din tabel sunt: CZ pentru controlerul de zbor, MT pentru modulul de telemetrie și SS pentru stația de sol.

Cod	Mesaj	Direcție	Rol
1	IMU	CZ → MT	Date brute inerțiale, pentru diagnosticarea vibrațiilor și a montajului
2	BATTERY	CZ → MT	Tensiunea acumulatorului
3	STATUS	CZ → MT	Cadrul afișat de tabloul de bord, trimis la 50 Hz
10	CONTROL	MT → CZ	Comenzile pilotului: propulsie, unghiuri dorite, viteză de rotație, indicatori
11	ARMING	MT → CZ	Cerere de armare sau oprire de urgență
12	MODE	MT → CZ	Modul de zbor: manual, menținere altitudine, menținere poziție
13	GAINS	MT → CZ	Coeficienți de reglare modificați de la tabloul de bord

Tabelul 4 – Tipurile de mesaje definite de protocol

4.3.3 DETECTAREA PIERDERII LEGĂTURII SERIALE

Cadrul de comandă pleacă de cincizeci de ori pe secundă indiferent dacă pilotul atinge sau nu ceva. Aceasta este o decizie de proiectare, nu o simplificare.

O implementare care ar trimite doar la schimbarea comenzii ar părea mai eficientă și ar dezarma drona în momentul în care pilotul încetează să apese butoane. Motivul este că receptorul nu poate distinge „nu am nimic de spus” de „legătura a murit”: ambele arată identic pe fir. Absența cadrului este singurul indiciu de defect disponibil, deci emisia trebuie să fie continuă.

Pe partea controlerului, watchdog-ul legăturii este definit astfel:

```
#define LINK_TIMEOUT_US 300000
```

flight_control.c:69

La o cadență de 50 Hz, adică un cadru la fiecare 20 ms, fereastra de 300 ms tolerează cincisprezece cadre pierdute consecutiv înainte să acționeze. Este suficient de lungă pentru a trece peste o întrerupere scurtă a legăturii radio și suficient de scurtă

pentru a conta: o dronă care continuă să execute ultima comandă primită timp de o secundă după pierderea legăturii este periculoasă într-un spațiu închis.

Pe partea modului de telemetrie există un al doilea nivel de protecție, pentru un caz pe care primul nu îl acoperă. Dacă browserul pilotului se închide sau telefonul se blochează, modulul de telemetrie continuă să trimită cadre perfect valide, iar watchdog-ul controlerului nu se declanșează niciodată — drona ar rămâne armată, în aer, fără nimeni care să o comande. De aceea modulul are propriile temporizări:

```
#define INPUT_TIMEOUT_US 500000 // 500 ms
#define DISARM_TIMEOUT_US 3000000 // 3 s
```

control_state.c:131–132

La 500 ms toate comenzile direcționale sunt eliberate, dar nivelul de propulsie se păstrează: drona se oprește din manevră și se nivelează, în loc să cadă. La 3 secunde se retrage cererea de armare, pe care controlerul o tratează imediat. Cele două praguri diferite reflectă două ipoteze diferite: prima este că browserul a rămas fără atenție, a doua este că nu se mai întoarce.

4.3.4 TRATAREA ERORILOR DE RECEPȚIE

Există două funcții de citire a unui cadru. Prima presupune că următorul octet din flux este începutul unui cadru și golește bufferul la orice eroare. A doua, folosită în producție, tratează fiecare eșec printr-o reluare: caută următorul octet 0xAA și încearcă din nou de acolo.

Diferența contează pe un flux continuu. Varianta care golește bufferul aruncă și cadrele bune deja aflate în coadă în spatele octetului corupt, transformând o singură eroare de transmisie în mai multe cadre pierdute. Pe traficul de tip cerere-răspuns strategia ar fi acceptabilă, dar pe un flux de 50 Hz este exact comportamentul de evitat (telemetry_uart.c:172–176).

4.4 CONTROLERUL DE ZBOR

4.4.1 IMPLEMENTAREA REGULATORULUI PID

Componenta `pid_controller` implementează cele trei soluții descrise teoretic în secțiunea 3.2. O iterație completă este o singură funcție, `pid_update()` (`pid_controller.c:80–146`).

Componentele proporțională și derivativă se calculează înaintea celei integrale, pentru că testul anti-windup are nevoie de ieșirea completă: trebuie să știe cât ar fi suma finală înainte de a decide dacă acest pas de integrare ar împinge sistemul și mai adânc în saturație.

Derivata calculată pe mărimea măsurată este soluția pentru problema modificării bruște a valorii dorite, descrisă în secțiunea 3.2.3:

```
d_raw = -(measurement - pid->prev_measurement) / dt;
```

pid_controller.c:99

Filtrarea derivatei se face cu un filtru trece-jos de ordinul întâi, cu coeficientul calculat o singură dată la inițializare, din frecvența de tăiere cerută și din perioada nominală a buclei:

```
const float rc = 1.0f / (2.0f * (float)M_PI * d_cutoff_hz);
pid->d_alpha = nominal_dt / (rc + nominal_dt);
```

pid_controller.c:70–71

Frecvențele de tăiere diferă de la un nivel la altul, pentru că zgomotul prezent la intrare diferă:

Bucă	Frecvență de tăiere	Motivul alegerii
Viteză unghiulară	60 Hz	Primește giroscopul brut, deci are nevoie de cea mai puternică filtrare
Unghi	30 Hz	Intrarea este deja netezită de estimatorul de orientare
Viteză și altitudine	10 Hz	Mărimile se schimbă lent, iar filtrarea agresivă nu introduce întârziere semnificativă la 50 Hz

Tabelul 5 – Frecvențele de tăiere ale filtrului derivatei (flight_control.c:177–179)

Integrarea condiționată. Este implementarea exactă a mecanismului descris în secțiunea 3.2.4:

```
const float candidate_integrator = pid->integrator + (pid->ki * error * dt);
const float candidate_output = p_term + candidate_integrator + d_term;
bool freeze_integrator = false;
if (candidate_output > pid->out_max && error > 0.0f) {
    freeze_integrator = true;
} else if (candidate_output < pid->out_min && error < 0.0f) {
    freeze_integrator = true;
}
```

pid_controller.c:109–118

Se observă condiția dublă: nu este suficient ca ieșirea candidat să depășească limita, ci trebuie ca și eroarea să ceară mai mult în aceeași direcție. O eroare care ar scoate bucla din saturație este integrată normal. Mecanismul corespunde soluției de integrare condiționată descrise în lucrarea citată [9].

Există două tipuri de resetare a reguletoarelor, folosite în situații diferite. Funcția `pid_reset()` șterge integratorul și istoricul derivatei, dar păstrează coeficienții; este calea folosită la armare și la dezarmare. Funcția `pid_set_gains()` schimbă coeficienții și păstrează integratorul; este calea folosită la reglarea în timpul zborului. Dacă a doua ar șterge integratorul, fiecare mișcare a unui cursor pe tabloul de bord ar produce o smucitură a dronei, ceea ce ar face reglarea în zbor imposibilă.

4.4.2 STRUCTURA ÎN CASCADĂ ȘI CELE OPT BUCLE

Cascada descrisă teoretic în secțiunea 3.3 este instanțiată ca opt reguletoare independente, configurate la inițializare (`flight_control.c:315–370`). Structura completă, cu mărimile care circulă între niveluri, este prezentată în figura următoare.

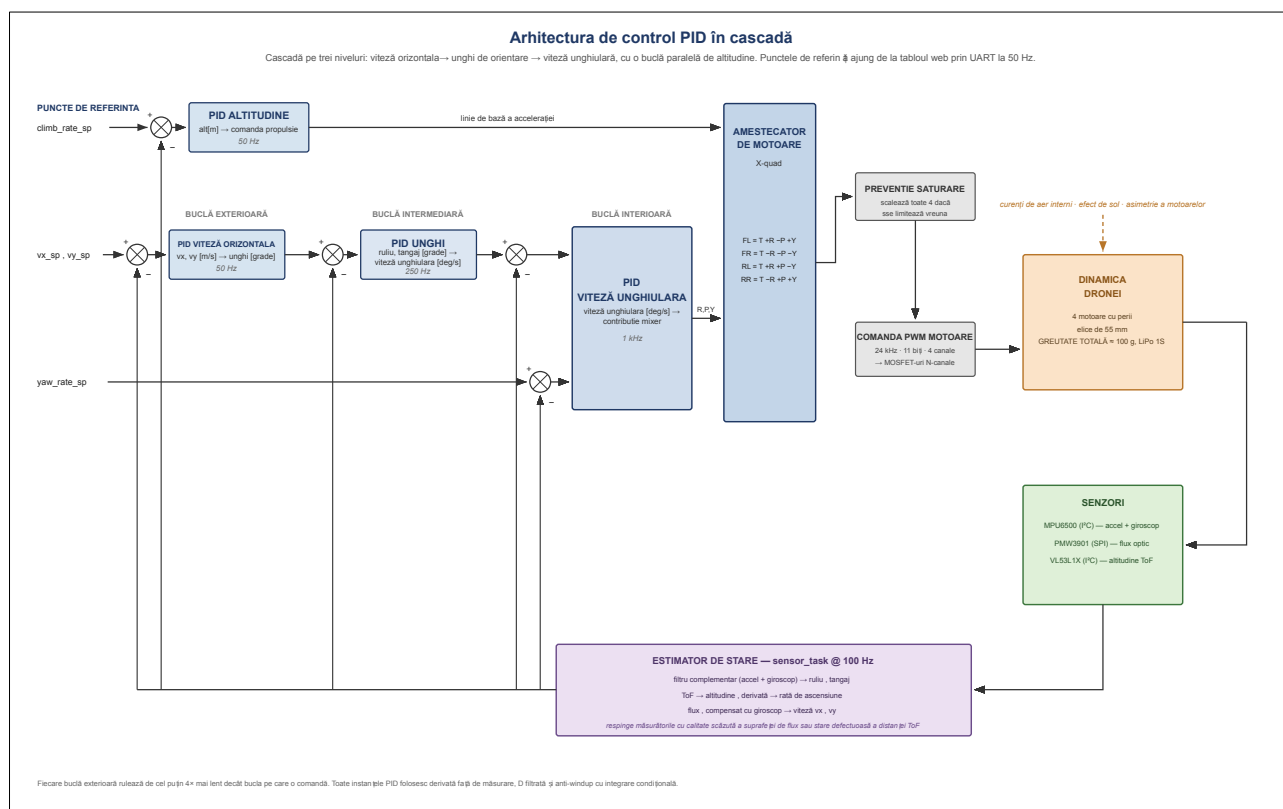


Figura 10 – Structura de reglare în cascadă, cu mărimile schimbate între niveluri

Cele opt bucle și limitele lor:

#	Bucă	Intrare → ieșire	Frecvență	Limite ieșire	Limită integrator
1–3	Viteză unghiulară (rulu, tangaj, girație)	grade/s → contribuție mixer	1000 Hz	$\pm 0,5$	0,2
4–5	Unghi (rulu, tangaj)	grade → grade/s	250 Hz	± 250 °/s	50
6–7	Viteză de translație (X, Y)	m/s → grade	50 Hz	$\pm 25^\circ$	10
8	Altitudine	metri → ajustare propulsie	50 Hz	$\pm 0,25$	0,25

Tabelul 6 – Cele opt bucle de reglare și limitele lor

Limita de ± 0.5 al buclelor interioare au fost alese astfel deoarece o singură axă nu poate revendica niciodată mai mult de jumătate din intervalul de comandă al motoarelor, ceea ce lasă loc celorlalte două axe și propulsiei. Fără această limitare, o corecție agresivă pe rulu ar putea consuma singură tot intervalul disponibil, iar comanda de tangaj ar rămâne fără efect exact în momentul în care este necesară.

Autoritatea nivelelor este limitată, fiecare nivel are voie să ceară nivelului de sub el doar atât cât este rezonabil fizic:

```
#define MAX_ANGLE_SETPOINT_DEG 25.0f // bucla de viteza nu poate cere mai
multa inclinare
#define MAX_RATE_SETPOINT_DPS 250.0f // bucla de unghi nu poate cere mai
multa rotatie
```

flight_control.c:146–147

Valoarea de 25 de grade este o înclinare din care o dronă de interior poate reveni fără să lovească un perete; 250 de grade pe secundă este o viteză de rotație pe care structura o poate atinge fără ca giroscopul să iasă din domeniul de măsurare configurat. Fără aceste limite, o eroare mare la un nivel exterior s-ar propaga ca o comandă imposibil de executat la nivelul de sub el, iar rezultatul ar fi saturație permanentă în loc de reglare.

Coeficienții de reglare a buclelor sunt în acest moment puncte de plecare, nu valori reglate. Procedura recomandată este cea din secțiunea 3.3, dinspre interior spre exterior:

```
#define DEFAULT_RATE_KP 0.0020f
#define DEFAULT_RATE_KI 0.0100f
#define DEFAULT_RATE_KD 0.00004f

#define DEFAULT_ANGLE_KP 4.5f // Ki = Kd = 0
```

```
#define DEFAULT_VELOCITY_KP 8.0f
#define DEFAULT_VELOCITY_KI 1.5f

#define DEFAULT_ALTITUDE_KP 0.40f
#define DEFAULT_ALTITUDE_KI 0.15f
#define DEFAULT_ALTITUDE_KD 0.05f
```

flight_control.c:160–174

Buclele de unghi au intenționat coeficienții integral și derivativ nuli: sunt regulatoare pur proporționale. O eroare staționară de unghi există, dar ea este preluată de bucla de viteză aflată deasupra, care comandă înclinarea necesară pentru a anula deriva — acesta este chiar avantajul structurii în cascadă. Adăugarea unei componente integrale aici ar introduce o a doua sursă de acumulare, în serie cu cea din bucla interioară, iar cele două ar interacționa într-un mod greu de reglat.

Coeficientul proporțional al buclei de unghi, 4,5, are o interpretare directă: o abatere de un grad față de unghiul dorit produce o viteză unghiulară dorită de 4,5 grade pe secundă pentru bucla de sub ea.

Toți coeficienții pot fi modificați în timpul zborului, prin mesajul GAINS de la tabloul de bord. Aceasta este proprietatea care face reglarea practicabilă: fără ea, fiecare încercare ar însemna aterizare, reprogramare și decolare, iar numărul de încercări posibile într-o sesiune ar scădea de la câteva zeci la câteva.

Girația nu are buclă de unghi. Nu există magnetometru, deci direcția absolută nu este observabilă. Butonul comandă direct o viteză de rotație, iar bucla interioară o urmărește (*flight_control.c:726–727*). Aceasta este o limitare acceptată în mod conștient: pentru o dronă de interior comandată vizual, direcția absolută nu este necesară.

4.4.3 ESTIMATORUL DE ORIENTARE

Aceasta componentă transformă eșantioanele brute ale unității inerțiale în date utilizabile de către buclele de control pentru a regla unghiul dronei. Ieșirea este o singură structură, actualizată la 1000 Hz:

Ieșire	Unitate de măsură	Cum se obține	Cine îl folosește
roll_rate, pitch_rate, yaw_rate	grade/s	direct de la giroscop	bucla de viteză unghiulară, 1 KHz
roll, pitch	grade	fuziune acc. + giroscop	bucla de unghi, 250 Hz
yaw	grade	integrarea giroscopului	pentru afișare
accel_valid	adimensional	starea ultimei actualizări	telemetrie, diagnostic
initialized	adimensional	condiție de armare	logica de armare

Tabelul 7 – Ieșirile estimatorului de orientare

Există o distincție importantă între vitezele unghiulare și unghiuri. Vitezele nu sunt estimate: sunt măsurate direct de giroscop și doar corectate cu offset-ul determinat la pornire. Bucla care ține efectiv drona în aer, cea de 1000 Hz, lucrează exclusiv pe aceste valori. Unghiurile, care rezultă dintr-o fuziune a celor doi senzori după principiul descris în secțiunea 3.4, ajung numai în bucla de unghi de 250 Hz. Consecința practică este că stabilitatea de bază depinde mai puțin de calitatea estimării unghiurilor față de calitatea estimării vitezelor unghiulare.

Calibrarea giroscopului se face prin colectarea a 1000 de eșantioane la interval de 2 ms, adică două secunde cu drona nemișcată, iar media pe fiecare axă devine offset-ul scăzut din toate citirile ulterioare (`imu_driver.c:207`). Fereastra nu este acceptată necondiționat: se urmăresc minimul și maximul pe fiecare axă, iar dacă diferența dintre acestea depășește `GYRO_CALIB_MAX_SPREAD_LSB` (200 LSB, adică aproximativ 12 grade pe secundă cu sensibilitatea de 16.4 LSB/g), fereastra este respinsă și procedura se reia, de cel mult trei ori (`imu_driver.c:256–261`). Motivul acestei verificări este de a asigura o calibrare cât mai precisă, eliminând posibile situații în care drona s-a mișcat în timpul calibrării. (`imu_driver.c:241–254`).

Calibrarea accelerometrului are rolul de „a netezi” valorile cu care pornește sistemul de control pentru a nu lua în calcul posibile erori de montare a senzorului sau un plan mai puțin orizontal de pe care drona va decola. Se face media a 500 de eșantioane și se rețin unghiurile de rulu și tangaj rezultate din vectorul gravitațional, care sunt apoi scăzute din unghiurile estimate (`imu_driver.c:286`).

4.4.4 ESTIMATORUL DE NAVIGAȚIE

Telemetrul laser măsoară distanța de la senzor până la suprafața de pe care fasciculul emis de acesta se reflectă. Când drona este înclinată se aplică următoarea corecție:

```
const float altitude_m = slant_range_m * cosf(roll_deg * DEG_TO_RAD)
                        * cosf(pitch_deg * DEG_TO_RAD);
```

nav_estimator.c:150

Aceasta reprezintă proiecția pe verticală a fasciculului și este aplicată doar până la 40 de grade înclinare (`nav_estimator.c:79`). Dincolo de această valoare, fasciculul, nu mai lovește podeaua de sub dronă, ci un perete sau un obiect lateral. Distanța returnată este reală, dar nu are nicio legătură cu altitudinea, iar modelul cosinusoidal nu poate detecta situația. Măsurătoarea este deci declarată invalidă, ceea ce dezactivează menținerea altitudinii și redă comanda pilotului.

Viteza pe orizontală. Se aplică relația din secțiunea 3.5.2, în trei pași distincți (nav_estimator.c:255–283):

```
// 1. conversie din unități ale senzorului în unități unghiulare
const float flow_x_rad = flow->delta_x / FLOW_COUNTS_PER_RAD;

// 2. se scade deplasarea unghiulară datorată rotației, măsurată de giroscop
const float translation_x_rad = flow_x_rad - rotation_x_rad;

// 3. conversie în metri pe secundă, folosind altitudinea
const float raw_velocity_x = (translation_x_rad / dt) * state.altitude;
```

nav_estimator.c:255, 277, 282

Pasul 2 este esențial. Senzorul nu poate distinge translația de rotație: o comandă de tangaj(*pitch*) derulează imaginea de sub dronă exact ca o deplasare înainte. Se scade deci deplasarea unghiulară pe care giroscopul a măsurat-o în același interval, iar ce rămâne este translația reală. Fără această corecție, orice înclinare comandată de bucla de viteză ar fi interpretată de aceeași buclă ca o deplasare, ceea ce ar produce o reacție pozitivă.

Dacă niciun eșantion bun nu ajunge în 200 de milisecunde, indicatorul de validitate corespunzător este retras (nav_estimator.c:88–89). Alternativa, păstrarea ultimei valori bune la nesfârșit, ar face bucla exterioară să regleze pe o altitudine care nu mai există. Mecanismul funcționează printr-o convenție simplă: o citire eșuată apelează totuși funcția de actualizare, dar cu un argument nul, ceea ce îi spune estimatorului „nu am date în acest ciclu” în loc folosească valoarea din iterația anterioară la nesfârșit.

Ambele conversii urmează practica din firmware-uri de zbor consacrate. Corecția cosinusoidală a distanței măsurate de telemetru apare sub aceeași formă în estimatorul PX4 [20]; ArduPilot aplică aceeași corecție [21].

Cei trei pași ai conversiei fluxului optic, trecerea în unități unghiulare, scăderea componentei de rotație măsurate de giroscop și scalarea cu altitudinea, reproduc procedura descrisă în lucrarea citată[16], pentru senzorul PX4FLOW, unde fluxul introdus de rotații este scăzut din valoarea măsurată, iar rezultatul este scalat la valoarea metrică presupunând o suprafață plană aflată la distanța indicată de telemetru.

4.4.5 MIXERUL

Funcția de mixare (flight_control.c:435–540) traduce cele patru comenzi ale cascadei factori de umplere pentru cele patru semnale PWM care comandă motoarele. Implementarea rezolvă problema descrisă în secțiunea 3.1.2 și este considerabil mai elaborată decât suma simplă prezentată acolo.

Pasul 0 — mixarea propriu-zisă, fără propulsie (`flight_control.c:440–443`).

Rezultatul este un set de valori cu semn, centrat în jurul lui zero. Ceea ce contează pentru atitudine este diferența dintre motoare, motiv pentru care propulsia se adaugă abia după ce se știe cât de mare este această diferență.

Pasul 1 — încadrarea în banda disponibilă. Banda utilizabilă este intervalul dintre pragul minim al motoarelor și valoarea maximă. Dacă diferența cerută de buclele de atitudine este mai largă decât banda, se scalează doar mixul, nu și propulsia:

```
const float available_band = 1.0f - MOTOR_IDLE_THRUST;
if (mix_range > available_band) {
    const float scale = available_band / mix_range;
    for (int i = 0; i < MOTOR_COUNT; i++) {
        mix[i] *= scale;
    }
    mix_scaled_latch = true;
}
```

flight_control.c:458–472

Motivul pentru care se scalează diferențele, și nu ieșirile finale, merită explicat. Scalarea ieșirilor, propulsie inclusă, ar micșora diferențele într-o proporție diferită de cea în care micșorează media, deci ar altera atitudinea comandată. Scalarea diferențelor păstrează direcția comenzii de atitudine și îi reduce doar mărimea, ceea ce este o degradare mult mai puțin dăunătoare.

Pasul 2 — plasarea propulsiei. Se calculează propulsia la care cel mai mic motor ajunge exact pe pragul minim, iar dacă propulsia cerută de pilot este sub aceasta, întregul grup este deplasat în sus:

```
const float lowest_allowed = MOTOR_IDLE_THRUST - mix_min;
const float wanted_boost = lowest_allowed - placed_throttle;
const float boost = clampf(wanted_boost, 0.0f,
MAX_MIXER_THROTTLE_BOOST);
placed_throttle += boost;
```

flight_control.c:479–490

Fără plafonul de 15%, acest mecanism ar deveni un regim în care mixerul ridică nelimitat media pentru a păstra autoritatea de control (`flight_control.c:131`). Pe o comandă de propulsie realizată cu butoane, asta ar însemna că apăsarea butonului de coborâre nu mai produce o coborâre reală, pentru că mixerul compensează, 15% este suficient pentru ca elicele să rămână în mișcare printr-o corecție, fără ca mixerul să se lupte cu pilotul.

Pasul 3 — restrângerea, dacă plafonul a intervenit. Dacă după deplasarea plafonată tot nu există loc, mixul este restrâns cu un factor aplicat identic pe toate cele trei axe (flight_control.c:514–532), astfel încât direcția comenzii se păstrează și doar mărimea scade.

Trei constante cu nume asemănătoare intervin în această zonă și este ușor de confundat rolul lor:

Constantă	Valoare	Rol
MOTOR_IDLE_THRESHOLD	0,05	Sub această propulsie comandată, motoarele sunt ținute complet oprite, chiar dacă drona este armată
MOTOR_IDLE_THRUST	0,06	Limita pe care mixerul o menține fiecărui motor atunci când drona este peste pragul de mai sus
MAX_MIXER_THROTTLE_BOOST	0,15	Cât are voie mixerul să ridice propulsia medie pentru a atinge limita

Tabelul 7 – Cele trei constante care intervin în comportamentul mixerului la propulsie mică

Prima are rolul de a împiedica acumularea în integratoare cât timp drona stă pe podea cu elicele abia învârtindu-se; a doua are rolul de a păstra autoritatea de control; a treia limitează efectul celei de-a doua asupra comenzii pilotului.

4.4.6 MAȘINA DE STĂRI DE ARMARE

Sistemul are trei stări. Tranzițiile sunt evaluate la fiecare iterație de 1 kHz (flight_control.c:544–616), nu la evenimente, ceea ce înseamnă că o condiție care dispare este observată în cel mult o milisecundă.

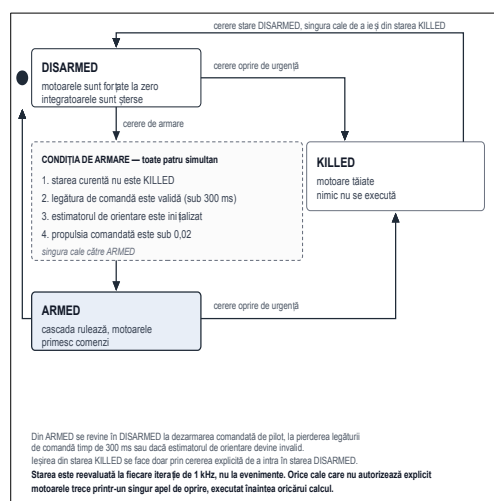


Figura 12 – Mașina de stări de armare și condițiile fiecărei tranziții (sursă: proprie)

Condiția de armare cere patru condiții simultan: starea curentă să nu fie cea de oprire de urgență, legătura de comandă să fie validă, estimatorul de orientare să fie inițializat și propulsia comandată să fie sub 0,02 (flight_control.c:72).

Oprirea de urgență este absolută și cu blocare. Odată primită, motoarele sunt tăiate și starea rămâne blocată. Starea se eliberează doar când pilotul a eliberat butonul de oprire și, simultan, nu mai cere armare (flight_control.c:566–572). Această a doua condiție este esențială: fără ea, eliberarea butonului ar putea, prin ea însăși, să repornească motoarele, ceea ce este exact comportamentul pe care o oprire de urgență trebuie să îl excludă.

Fără ștergerea integratoarelor la armare și la dezarmare, o valoare acumulată în timp ce drona stătea pe podea, ar fi aplicată instantaneu la decolare. Din același motiv, contorul care generează frecvențele celor trei bucle este readus la zero, astfel încât următoarea armare să înceapă cu toate cele trei bucle rulând în aceeași iterație.

4.4.7 DRIVERUL DE MOTOARE

Cele patru canale PWM sunt generate de periferia LEDC a microcontrolerului [22].

```
#define MOTOR_PWM_FREQ_HZ          24000
#define MOTOR_PWM_RESOLUTION_BITS  LEDC_TIMER_11_BIT
#define MOTOR_PWM_MAX_DUTY          ((1u << 11) - 1u) // 2047
```

motor_driver.c:45–47

Periferia produce frecvența dintr-un ceas de 80 MHz, iar frecvența maximă la o rezoluție dată este raportul dintre acest ceas și numărul de trepte. La 11 biți, plafonul hardware este $80 \cdot 10^6 / 2048 \approx 39$ kHz, deci 24 kHz este sub această limită. În sens invers, 24 kHz a fost aleasă pentru că este peste limita superioară a auzului uman, de aproximativ 20 kHz, astfel încât comutarea nu produce fluierat audibil, și în același timp suficient de joasă încât un tranzistor de mici dimensiuni să comute curat, fără pierderi mari. Frecvențe de comandă de acest ordin sunt uzuale și în programele de control consacrate pentru drone cu motoare cu perii, unde parametrul este configurabil tocmai din aceste motive [23].

Conversia din propulsie normalizată în factor de umplere este liniară și cu limitare:

```
static uint32_t motor_thrust_to_duty(float thrust) {
    thrust = clampf(thrust, 0.0f, 1.0f);
    return (uint32_t)lroundf(thrust * (float)MOTOR_PWM_MAX_DUTY);
}
```

motor_driver.c:167–171

Aceasta este limitarea finală discutată în secțiunea 3.1.2: o comandă negativă devine factor de umplere zero. Mixerul descris anterior are exact rolul de a face ca situația să apară cât mai rar.

4.4.8 MONITORIZAREA ACUMULATORULUI

Tensiunea acumulatorului este măsurată printr-un divizor rezistiv și convertorul analog-digital (battery_monitor.c:23–43). Inițializarea componentei nu este critică pentru pornirea sistemului: dacă inițializarea eșuează, programul continuă și raportează o valoare nominală. Argumentul este că tensiunea acumulatorului este telemetrie, nu control, nimic din cascadă nu o citește. Deci un convertor defect sau un divizor nemontat nu trebuie să împiedice zborul (main.c:496–500).

4.4.9 ORDINEA DE ÎNȚIALIZARE

Ordinea operațiilor din funcția principală (main.c:461–583) este impusă de două constrângeri, nu de preferință.

În primul rând, motoarele se configurează înaintea oricărui alt lucru (main.c:479). Niciun cod nu are voie să poată comanda propulsie înainte ca cele patru canale să existe și să fie duse la zero.

În al doilea rând, telemetrul laser se atașează pe magistrala I2C pe care o creează inițializarea unității inerțiale, iar aceasta rulează în interiorul buclei de control, nu în funcția principală. Fără sincronizare, sarcina de senzori ar încerca să folosească o magistrală inexistentă sau ar concura cu inițializarea pentru ea. Soluția este un semafor binar:

```
if (xSemaphoreTake(imu_ready_semaphore, pdMS_TO_TICKS(30000)) != pdTRUE) {  
    ESP_LOGE(TAG, "IMU never became ready - navigation sensors will not be  
    started");  
}
```

main.c:562–564

Timpul de așteptare de 30s este calculată, astfel (main.c:555–561): calibrarea accelerometrului parcurge 500 de eșantioane la 2 ms, iar cea a giroscopului 1000 de eșantioane, cu până la trei reîncercări dacă detectează mișcare. Cu întârzierea rotunjită la tactul de 1 kHz și cu circa 0,5 ms de citire I2C pe eșantion, fiecare baleiaj este mai aproape de 3 secunde decât de 2, deci aproximativ 11 secunde în cazul cel mai defavorabil. Treizeci de secunde lasă marjă fără ca telemetrul să pornească vreodată în timpul calibrării.

Unitatea inerțială, driverul de motoare, estimatorul de orientare și controlul zborului sunt critice pentru sistem: fără ele nu se poate zbura, iar programul se oprește. Monitorul de acumulator, telemetrul laser și senzorul de flux optic permit pornirea sistemului în caz că nu sunt prezente componentele hardware necesare sau inițializarea lor eșuează. În cazul din urmă, se afișează un avertisment și programul trece mai departe cu buclele exterioare dezactivate.

4.5 MODULUL DE TELEMETRIE

4.5.1 SINTEZA COMENZILOR PILOTULUI

Tabloul de bord rulează într-un browser, pe telefon sau un laptop. Problema de proiectare este că o pagină web nu are manșe fizice, iar comanda unei drone este o mărime proporțională.

Prima implementare folosea butoane pentru toate axele, cu rampă la apăsare și revenire la eliberare. S-a dovedit a fi foarte complicat de navigat drona folosind aceste butoane așa că soluția actuală este o manșă analogică pentru ruliu și tangaj pe ecranul tactil, unde poziția degetului este unghiul comandat. Girația și propulsia păstrează butoanele, pentru că sunt mărimi pe care pilotul le setează și le lasă, nu le ajustează continuu.

Deplasarea normalizată a manșei trece printr-o funcție de forma $(1 - E) \cdot x + E \cdot x^3$ înainte de scalarea în grade, cu $E = 0,60$ (control_state.c:93). Efectul este rezoluție fină în apropierea centrului, exact acolo unde se fac corecțiile de derivă:

Deplasarea manșei	Cu curbă exponențială	Cu răspuns liniar
30 %	1,4°	3,0°
50 %	2,8°	5,0°
100 %	10°	10°

Tabelul 8 – Efectul curbei exponențiale asupra unghiului comandat

La deplasări mici, comanda este de aproximativ două ori mai fină decât ar fi cu un răspuns liniar, fără nicio pierdere de autoritate la capătul cursei.

Limitarea vitezei de variație a vitezei unghiulare este de 120 de grade pe secundă (control_state.c:94). Ea există pentru două cazuri anormale: o valoare coruptă sau intenționat greșită care ar sări peste toată scala într-un singur cadru, și watchdog-ul de browser care aduce manșa la zero.

Limitele axelor (control_state.c:57–58, 112):

```
#define MAX_ROLL_PITCH_DEG 10.0f // drona zboara in interior, langa obiecte
#define MAX_YAW_RATE_DPS 90.0f
#define THROTTLE_MAX 0.85f // rezerva pentru buclele de atitudine
```

control_state.c:57, 58, 112

Plafonul de propulsie la 0,85 lasă 15 % rezervă pentru ca mixerul să aibă unde adăuga corecțiile de atitudine. Controlerul de zbor limitează încă o dată pe partea lui, deci valorile de aici sunt limitele de confort, nu limitele de siguranță, o placă nu are încredere în valorile primite de la cealaltă.

4.5.2 PUNCTUL DE ACCES WI-FI

Componenta este restrânsă, iar aproape toată valoarea ei stă în două setări care diferă de configurația implicită.

Dezactivarea economiei de energie:

```
esp_wifi_set_ps(WIFI_PS_NONE); // obligatoriu DUPA esp_wifi_start()
```

wifi_ap.c:154

Implicit, ESP-IDF lasă radioul Wi-Fi să intre în repaus între două momente de sincronizare cu punctul de acces, iar cadrele primite între timp sunt păstrate până la următoarea trezire [24]. Trezirile au loc la aproximativ 100 de milisecunde una de alta, deci un mesaj poate aștepta până la acest interval înainte să ajungă la aplicație. Tabloul de bord trimite comenzi la 20 Hz, adică una la fiecare 50 de milisecunde. O întârziere suplimentară de până la 100 de milisecunde, diferită de la o comandă la alta, face ca acestea să sosească grupat și inegal: drona nu mai reacționează în momentul în care pilotul mișcă manșa, ci cu o pauză variabilă, ceea ce o face practic imposibil de pilotat. Din acest motiv economia de energie a radioului este dezactivată explicit.

Reducerea puterii de emisie (*wifi_ap.c:176*): 13 dBm, în loc de maximul radioului. Emisia este cel mai mare consumator de curent al radioului, iar drona zboară într-o cameră deci nu are nevoie de o rază de acțiune foarte mare. Este important de observat că acesta nu este același compromis ca economia de energie: dezactivarea repausului păstrează latența mică, în timp ce puterea de emisie schimbă doar raza de acțiune, nu și timpul de răspuns.

4.5.3 SERVERUL HTTP ȘI TABLOUL DE BORD

Serverul expune nouă puncte de acces, dintre care unul singur se află pe calea critică: cererea de intrare, la 20 Hz. Componenta nu deține stare proprie: fiecare handler traduce cererea într-un apel către altă componentă și se întoarce imediat.

Fiecare handler de scriere se adaugă într-o coadă. Acesta este tiparul care menține calea de comandă receptivă:

Punct de acces	Ce face efectiv
POST /api/input	Memorează starea butoanelor și poziția manșei; integrarea rulează pe sarcina de 50 Hz
POST /api/capture	Pune un jeton într-o coadă; nu capturează imaginea și nu scrie fișierul
POST /api/gains	Pune coeficienții într-o coadă; cadrul pleacă la următorul ciclu de 50 Hz
POST /api/preview	Setează un indicator; sarcina de capturare aplică schimbarea de mod
POST /api/arm	Armare, dezarmare sau oprire de urgență; șterge și o oprire în așteptare
GET /api/status	Returnează starea completă, ca un singur obiect JSON
GET /preview.jpg	Un cadru al vizorului, ca imagine JPEG

Tabelul 9 – Punctele de acces HTTP și tratarea lor

Motivul acestui tipar este direct: o scriere pe cardul SD executată în interiorul unui handler HTTP ar bloca serverul, ceea ce ar bloca cererile de intrare de 20 Hz, ceea ce ar declanșa watchdog-ul de browser în plin zbor. Punerea în coadă rupe acest lanț.

Tabloul de bord este compilat în binar, prin directiva de includere a fișierului HTML în imaginea firmware-ului. Consecințele sunt două, ambele intenționate: pagina funcționează fără card SD montat și nu poate rămâne în urma firmware-ului. Nu există posibilitatea ca o pagină veche să comande un firmware nou, ceea ce elimină o întreagă categorie de erori. Antetul care interzice memorarea în cache completează protecția: un tablou de bord servit dintr-o versiune veche păstrată de browser, care comandă o dronă reală, este un mod de defectare serios.

Costul acestei alegeri este că modificarea paginii cere reprogramarea plăcii. Pentru dezvoltare există un server auxiliar scris în Python, care servește același fișier direct de pe disc, ceea ce face costul irelevant în faza de proiectare a interfeței.

4.5.4 SUBSISTEMUL DE CAPTURARE A IMAGINILOR

Componenta este complet independentă de zbor. Nimic din ea nu poate opri drona, și nimic din ea nu rulează pe același nucleu cu calea de comandă.

Expunerea, balansul de alb și amplificarea sunt blocate. Aceasta este decizia care leagă subsistemul de scopul declarat în subcapitolul 1.2. Expunerea automată este activ dăunătoare pentru reconstrucție tridimensională: pe măsură ce drona se rotește în jurul obiectului, măsurarea automată se reface la fiecare cadru, iar aceeași suprafață iese cu o luminozitate diferită în fiecare fotografie. Programele de reconstrucție interpretează aceasta ca pe o schimbare a scenei, nu a camerei. Ghidurile de capturare pentru fotogrammetrie recomandă din acest motiv setări fixe de expunere pe toată durata unei sesiuni [25].

4.6 ARHITECTURA HARDWARE A SISTEMULUI

Schema electrică a dronei este prezentată în figura de mai jos. Ea cuprinde cele două module de calcul, cei trei senzori, etajul de comandă a motoarelor, divizorul rezistiv de măsurare a acumulatorului și sursa de alimentare.

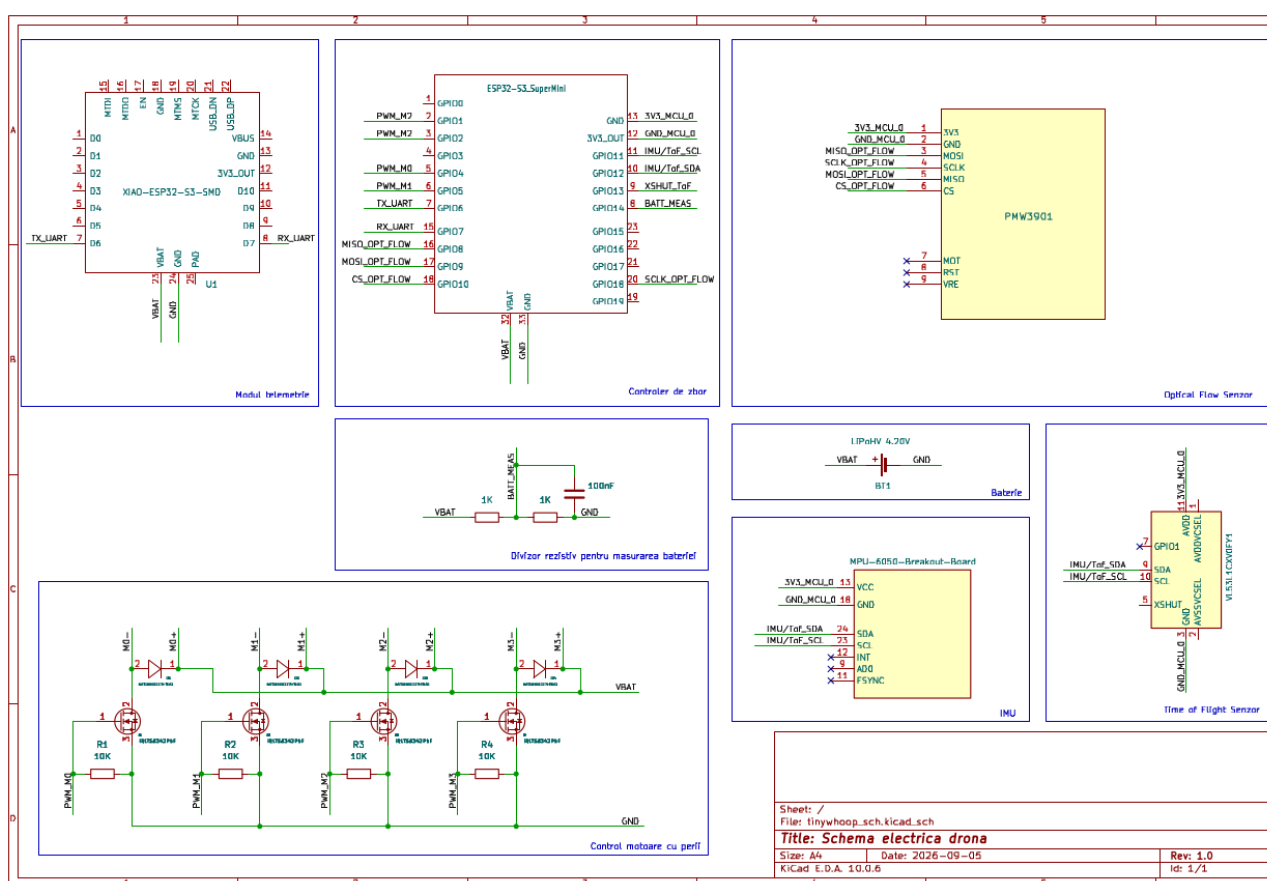


Figura 13 – Schema electrică a sistemului (sursa: proprie)

4.6.1 ALIMENTAREA

Sursa este un singur acumulator LiPo HV de 4,20 V cu o singură celulă (1S). Din el se alimentează direct, fără etaj intermediar, atât cele două microcontrolere, fiecare având propriul regulator intern de 3,3 V, cât și motoarele.

Faptul că motoarele și logica împart aceeași sursă are o consecință care apare de mai multe ori în această lucrare: la fiecare variație bruscă a comenzii de propulsie, tensiunea pe magistrala VBAT scade momentan, iar aceeași comandă produce o tracțiune mai mică pe măsură ce acumulatorul se descarcă. Nu există compensare de tensiune în firmware, pentru că relația dintre tensiune și tracțiune nu a fost măsurată; efectul lent este absorbit de componentele integrale ale cascadei, iar cel rapid rămâne o sursă de zgomot asupra căreia sistemul nu acționează.

Tensiunea de alimentare este măsurată printr-un divizor rezistiv format din două rezistențe egale de 1 k Ω , cu un condensator de 100 nF legat între intrarea convertorului analog-digital și masă. Raportul de divizare este deci 2:0, valoare care apare identic în firmware. Condensatorul are rolul de a minimiza zgomotul pe linia de măsurare iar valoarea utilizată este recomandată în documentația convertorului analog-digital [26].

4.6.2 REPARTIZAREA TERMINALELOR CONTROLERULUI DE ZBOR

Ieșirile/ intrările utilizate împreună cu componentele la care sunt legate sunt după cum urmează:

- GPIO1, GPIO2, GPIO4, GPIO5 reprezintă ieșirile de PWM ale celor patru motoare
- GPIO6, GPIO7 reprezintă liniile portului serial TX_UART și RX_UART pe controlerul de zbor
- GPIO8, GPIO9, GPIO10, GPIO18 reprezintă semnalele MISO, MOSI, CS, SCLK utilizate de magistrala SPI necesară senzorului de flux optic
- GPIO11, GPIO12 SCL, SDA Magistrala I²C partajată de IMU și telemetrul laser
- GPIO13 reprezintă comanda de oprire a telemetrului laser
- GPIO14 reprezintă intrarea analogică pentru divizorul acumulatorului
- GPIO 43, GPIO44 reprezintă liniile portului serial TX_UART și RX_UART pe modulul de telemetrie

Repartizarea nu este arbitrară. Terminalele GPIO0, GPIO3, GPIO45 și GPIO46 sunt terminale de configurare a modului de pornire pe ESP32-S3, iar documentația de proiectare hardware a producătorului cere ca starea lor să nu se modifice în momentul alimentării [27].

4.6.3 MAGISTRALELE SENZORILOR

Cei trei senzori nu partajează o singură magistrală, iar împărțirea urmează interfața fiecăruia.

Unitatea inerțială și telemetrul laser sunt amândouă pe magistrala I²C, la adrese diferite, deci un singur set de două terminale deservește doi senzori. Terminalul XSHUT al telemetrului este comandat separat, din GPIO13. Rolul lui este dublu: permite ținerea senzorului în stare de oprire în timpul inițializării, astfel încât să nu perturbe magistrala înainte de a fi configurat, și oferă singura cale de a-l readuce la o stare cunoscută dacă rămâne blocat, fără a întrerupe alimentarea întregii plăci. Terminalele de întrerupere ale

ambilor senzori rămân neconectate: firmware-ul le citește periodic, la frecvență fixă, deci nu are nevoie de semnalizare asincronă.

Senzorul de flux optic are o magistrală SPI proprie, cu patru terminale. Alegerea nu este o preferință de proiectare, ci o constrângere: circuitul PMW3901 nu expune o interfață I²C. Separarea are însă și un avantaj: transferurile lui, care sunt cele mai frecvente dintre cei trei senzori, nu concurează cu I²C-ul pe care rulează bucla de citire a unității inerțiale.

4.6.4 CIRCUITUL DE COMANDĂ A MOTOARELOR

Cele patru motoare cu perii sunt comandate identic, fiecare printr-o celulă formată din trei componente: un tranzistor MOSFET cu canal N de tip IRLTS6342, o diodă Schottky BAT165 și o rezistență de 10 k Ω .

Comutarea se face pe partea de masă. Motorul este legat între VBAT și drena tranzistorului, iar sursa la masă. Această topologie este cea mai simplă posibilă pentru o comandă unidirecțională: poarta tranzistorului se află la potențial apropiat de masă, deci semnalul de 3,3 V al microcontrolerului o poate comanda direct, fără circuit intermediar de ridicare a nivelului. Tranzistorul ales este de tip „logic level”, adică proiectat să conducă complet la o tensiune de comandă de 3,3 V, o condiție esențială, pentru că un tranzistor comandat parțial disipă putere în loc să comute.

Dioda antiparalelă este legată cu anodul la drenă și catodul la VBAT. Ea nu conduce în timpul funcționării normale. Rolul ei apare la fiecare blocare a tranzistorului: înfășurarea motorului este o inductanță, iar la oprirea acestuia motorului, până când acesta se oprește, acționează ca un generator, astfel producând un curent care are nevoie de o cale pe unde să nu afecteze restul circuitului. Dioda oferă o astfel de cale pentru curentul generat. Alegerea unei diode Schottky, și nu a uneia obișnuite, este motivată de frecvență: timpul ei de revenire este mult mai scurt, deci pierderile de comutație rămân mici.

Rezistența de 10 k Ω este legată între poarta tranzistorului și masă. Terminalele microcontrolerului sunt în stare de înaltă impedanță în intervalul dintre alimentare și configurarea lor de către firmware. Fără această rezistență, poarta ar rămâne flotantă în acel interval, iar sarcina acumulată pe ea ar putea porni motorul înainte ca sistemul să fie inițializat. Rezistența forțează poarta la masă și garantează că la alimentare și la orice defect care lasă terminalul neconfigurat motoarele sunt oprite.

Această celulă are o limitare inherentă, discutată la capitolul de concluzii: comanda este unidirecțională și fără reacție. Nu se poate frâna activ motorul și nu se cunoaște turația reală — bucla de control comandă un factor de umplere și presupune tracțiunea rezultată.

4.6.5 LEGĂTURA DINTRE PLĂCI

Cele două module sunt legate prin două fire de date și masa comună: GPIO6 și GPIO7 ale controlerului de zbor către terminalele D6 și D7 ale modulului de telemetrie. Faptul că sunt două fire independente, unul pentru fiecare sens, are o consecință practică descoperită în utilizare: cele două senzuri pot ceda separat, iar un singur fir întrerupt produce o legătură funcțională într-o singură direcție.

4.7 INSTRUMENTE DE ANALIZĂ ȘI VALIDARE

4.7.1 STADIUL STAȚIEI DE SOL

Stația de sol este o aplicație scrisă în Python, cu interfață grafică, care rulează pe calculator și afișează în timp real interiorul oricărei bucle PID selectate: valoarea dorită, valoarea măsurată, cele trei componente separat, ieșirea și cei doi indicatori de saturație. Aceasta este implementată și funcțională ca mecanism: legătura UDP transportă traficul de reglare, aplicația afișează interiorul buclei selectate și permite modificarea coeficienților în timpul zborului.

Cu toate acestea, datele returnate de acest instrument nu au ajuns încă într-un stadiu în care să se poată folosi cu încredere în procesul de testare și validare al sistemului. Din acest motiv nu au fost folosite ca bază pentru nicio concluzie cantitativă din această lucrare. Verificarea corespondenței dintre ce afișează aplicația și ce calculează efectiv controlerul rămâne de făcut, iar până atunci instrumentul este util pentru observarea calitativă a comportamentului unei bucle, nu pentru măsurători. Acesta este și motivul pentru spațiul redus care i-a fost acordat în restul lucrării.

4.7.2 TRASEUL DATELOR DE DEPANARE

Bucula de control publică eșantioane într-o coadă care poate stoca maxim 64 de elemente (`pid_registry.c:44`), folosind un timeout zero:

```
if (xQueueSend(g_debug_queue, &sample, 0) != pdTRUE) { ... }
```

pid_registry.c:308

Pierderea eșantioanelor este o decizie de proiectare, nu un defect: o buclă de 1 kHz nu are voie să aștepte după un consumator de 50 Hz. Sarcina de telemetrie preia până la 32 de eșantioane pe ciclu de 20 de milisecunde (`main.c:55`), ceea ce acoperă 1600 de eșantioane pe secundă, confortabil peste cei 500 Hz pe care îi cere stația de sol la setarea maximă. Coada se umple doar dacă legătura serială devine ea însăși limitarea.

4.7.3 GENERATORUL DE SEMNAL DE TEST

Mesajul `PID_INJECT` permite injectarea unui semnal cunoscut în valoarea dorită a unei bucle, cu o bază de timp comună tuturor buclelor care rulează cu același tact (`flight_control.c:812–816`). Aceasta face posibilă observarea răspunsului la un impuls

controlat și repetabil, în loc să se aștepte ca zborul să producă întâmplător o perturbație potrivită. Este diferența dintre o măsurătoare și o observație.

4.7.4 ÎNREGISTRAREA ZBORURILOR

Stația de sol are și funcționalitatea de a înregistra jurnale de zbor. Formatul ales este JSON delimitat prin linii noi. Motivul pentru care există această funcționalitate este că un astfel de jurnal poate fi valoros în urma unui zbor cu probleme; ar putea să ajute la identificarea cauzei problemei întâmpinate.

Înregistrarea nu se face comprimat: un flux comprimat oprit brusc ar putea lăsa un fișier complet ilizibil. Comprimarea se poate face după zbor, iar cititorul deschide transparent fișierele comprimate.

Costul acestei alegeri este dimensiunea: aproximativ 1,5 MB pe minut de zbor. Un zbor de zece minute înseamnă 15 MB, ceea ce este acceptabil pe disc și suficient de mic încât fișierul să poată fi transmis întreg.

4.8 TESTAREA ȘI VERIFICAREA

Un sistem în care o eroare de semn duce la răsturnarea aeronavei la prima decolare nu poate fi testat prin încercări repetate de zbor. Au fost construite deci mai multe mecanisme și bancuri de testare prin care ipotezele pot fi verificate înainte de a fi puse la încercare în aer.

4.8.1 PROIECTE DE CALIBRARE SEPARATE

Există două proiecte ESP-IDF independente, dedicate câte unui senzor. Ele folosesc aceleași drivere. Astfel, ceea ce se măsoară pe banc este exact codul care va zbura, iar o corecție făcută în timpul calibrării ajunge automat în firmware-ul de zbor. Fiecare proiect pornește un singur senzor și afișează citirile pe portul serial, ceea ce elimină din ecuație orice interacțiune cu restul sistemului.

4.8.2 MODUL DE MONITORIZARE CU DRONA DEZARMATĂ

Semnul buclei care transformă eroarea de viteză în unghi comandat este cel mai periculos parametru neverificat din sistem: un semn greșit acolo înseamnă reacție pozitivă, deci drona accelerează departe de punctul de menținere, în loc să se stabilizeze.

Până la introducerea acestui mod, singura cale de a verifica semnul era angajarea menținerii poziției în zbor și observarea rezultatului, o metodă anevoioasă de a afla răspunsul.

Soluția (flight_control.c:108, 774–790): cu drona dezarmată, rulează exclusiv bucla exterioară și publică starea ei internă, motoarele fiind deja forțate la zero de invarianța din secțiunea 4.4.6. Verificarea se face cu drona în mână: se ține la 20–50 cm deasupra unei

podele cu textură, se apasă butonul de menținere, se deplasează lateral și valorile din tabloul de bord. Ieșirea buclei este unghiul pe care ar vrea să îl comande, deci trebuie să se opună direcției de deplasare.

4.8.3 INDICATORII DE DIAGNOSTIC DIN CADRUL DE STARE

Opt biți din cadrul de stare (telemetry_uart.h:68–95) raportează condiții care altfel nu ar fi observabile din exterior:

Indicator	Ce semnalează
LINK_OK	Watchdog-ul legăturii de comandă este satisfăcut
ATTITUDE_INIT	Filtrul complementar s-a stabilizat; armarea este permisă
ALT_VALID	Estimarea de altitudine este de încredere
VEL_VALID	Estimarea de viteză pe orizontală este de încredere
KILLED	Starea oprire de urgență este activă
MIX_BOOST_CAP	Mixerul a avut nevoie de mai multă rezervă de propulsie decât permite plafonul
MIX_SCALED	Comanda de atitudine a fost redusă pentru a încăpea în banda motoarelor
ACCEL_REJECTED	Accelerometrul a fost eliminat din fuziune de poarta de validitate

Tabelul 10 – Indicatorii de diagnostic transmiși în cadrul de stare

4.8.4 MODUL DE BANC PENTRU MOTOARE

Pentru măsurarea pragului minim de comandă al motoarelor, programul principal are un mod compilabil separat (main.c:104) în care aplicația pornește exclusiv driverul de motoare și nimic altceva: niciun senzor, niciun estimator și nicio buclă de control nu poate atinge motoarele în timpul măsurării. Cu elicele montate, fiecare motor este dus în trepte de la zero până la valoarea la care pornește din repaus, iar rezultatul este valoarea celui mai defavorabil motor, plus o marjă. Rezultatele sunt prezentate în capitolul 5.

4.8.5 VERIFICAREA FRECVENȚĂ BUCLEI DE CONTROL

Prima verificare, și cea mai simplă, este că bucla interioară rulează efectiv la frecvența la care a fost proiectată. Tot ce urmează depinde de aceasta: intervalul de timp folosit de estimator, de componenta integrală și de cea derivativă este cel nominal, iar dacă bucla rulează la altă frecvență, toate constantele de timp din sistem sunt greșite cu același factor.

Bucula își numără propriile iterații pe ferestre de o secundă și publică rezultatul în cadrul de stare (`flight_control.c:229–231`). Verificarea constă în urmărirea acestei valori în tabloul de bord, în condiții realiste: cu toți senzorii activi, cu telemetria pornită și cu ambele nuclee încărcate.

4.8.6 CE A RĂMAS NEVERIFICAT

Prezentarea onestă a stării sistemului este mai utilă decât o descriere care ar sugera un produs finalizat. Următoarele elemente sunt marcate ca neverificate direct în codul sursă, prin comentarii care descriu și procedura de determinare:

Element	Stare	Referință în cod
HOVER_THROTTLE = 0,45	Estimare	<code>flight_control.c:143</code>
FLOW_COUNTS_PER_RAD = 500	Estimare din specificația senzorului	<code>nav_estimator.c:75</code>
Coeficienții PID	Puncte de plecare, nereglate în zbor	<code>flight_control.c:150–174</code>
ACCEL_LPF_CUTOFF_HZ = 15	Valoare rezonabilă, nemăsurată	<code>attitude_estimator.c:66</code>

Tabelul 11 – Elementele rămase neverificate la momentul redactării

5 DESCRIEREA ȘI UTILIZAREA PRODUSULUI

5.1 LIVRABILELE PROIECTULUI

Proiectul livrează două firmware-uri independente, un protocol comun, un set de unelte de banc și implementarea fizică a dronei atât din punct de vedere hardware cât și software.

Livrabil	Conținut	Rulează pe
flight_controller/	Bucula de control în cascadă, estimatoarele, mixerul, driverele de senzori și de motoare	ESP32-S3 Super Mini
telemetry_module/	Punct de acces Wi-Fi, tablou de bord web, sinteza comenzilor, capturarea imaginilor	XIAO ESP32-S3 Sense
shared_components/ telemetry_uart/	Protocolul binar dintre plăci: cadre, CRC-16, tipuri de mesaje	Ambele, o singură copie pe disc
tools/ground_station/	Aplicație desktop pentru reglarea buclelor în zbor	Calculator, Python + PyQt6
tof_calibration/ flow_calibration/	Proiecte de banc pentru determinarea constantelor celor doi senzori de navigație	ESP32-S3 de rezervă
Mod de banc pentru motoare	Compilat în firmware-ul de zbor, exclude orice altă componentă	ESP32-S3 Super Mini

Tabelul 13 – Livrabilele proiectului

5.2 PUNEREA ÎN FUNCȚIUNE

Nu există o compilare comună: fiecare firmware se construiește separat, din rădăcina fiecărui proiect, astfel:

```
idf.py set-target esp32s3
idf.py build flash
idf.py monitor (după caz)
```

Componenta partajată telemetry_uart este preluată în ambele proiecte prin câmpul EXTRA_COMPONENT_DIRS folosit în CMake, deci nu trebuie copiată, o modificare a protocolului ajunge automat în ambele firmware-uri la următoarea compilare. Ordinea de

programare nu contează, dar ambele plăci trebuie să ruleze versiuni construite din același protocol; verificările de dimensiune din `telemetry_uart.h` opresc compilarea dacă structurile au divergențe.

La alimentare, modulul de telemetrie pornește punctul de acces și tipărește în consolă numele rețelei și adresa tabloului de bord. Pilotul se conectează cu telefonul sau laptopul la rețea și deschide `http://192.168.4.1/`. Pagina este compilată în firmware, deci nu necesită conexiune la internet și nici un server extern.

În paralel, controlerul de zbor execută calibrarea automată descrisă în 4.4.3: aproximativ două secunde pentru offsetul giroscopului și o jumătate de secundă pentru offsetul de montare al accelerometrului. Drona trebuie să stea nemișcată, pe o suprafață orizontală, în tot acest interval. Dacă este mișcată, procedura de giroscop respinge fereastra și se reia, de cel mult trei ori.

6 CONCLUZIA

Lucrarea a pornit de la obiectivul de a realiza, de la zero, sistemul de pilotare al unei drone de mici dimensiuni construite integral în cadrul proiectului, fără a folosi un firmware de zbor existent, și de a face acest lucru pe un microcontroler ESP32-S3, adică pe o platformă care nu este cea obișnuită pentru astfel de aplicații. Obiectivul a fost atins în sensul că sistemul este complet ca structură și funcțional ca mecanism: au fost realizate o buclă de reglare în cascadă cu opt regulatoare, executată la 1 kHz pe un nucleu dedicat, un estimator de orientare cu calibrarea automată a giroscopului și a accelerometrului la pornire, un estimator de navigație care combină telemetrul laser cu senzorul de flux optic, un mixer care tratează explicit saturarea motoarelor, o mașină de stări de armare cu oprire de urgență, un protocol binar propriu cu verificare de integritate și supraveghere a legăturii în ambele sensuri între cele două plăci, un tablou de bord servit direct de aeronavă și un set de instrumente de diagnostic și de banc. Ceea ce nu a fost atins este reglarea sistemului: coeficienții reglatoarelor rămân puncte de plecare, iar câteva constante fizice sunt încă estimări. La acestea se adaugă limitări inerente soluției alese, dintre care cea mai importantă vine din folosirea motoarelor cu perii care au un comportament neliniar cu privire la factorul de umplere comandat și nivelul propulsiei rezultat, precum și faptul că stația de sol, deși funcțională ca mecanism, nu produce încă date pe care să se poată sprijini o concluzie cantitativă. Contribuția proprie constă în proiectarea și implementarea celor două firmware-uri și a protocolului dintre ele, în proiectarea circuitelor hardware, al șasiului și a cablajului. Dezvoltarea ulterioară urmează o ordine impusă de dependențe: efectuarea măsurătorilor de banc rămase, validarea stației de sol prin compararea valorilor afișate cu cele calculate efectiv de controler, reglarea buclelor dinspre interior spre exterior și calibrarea senzorilor de navigație.

Bibliografie

- 1: ***, , , <https://www.sciencedirect.com/science/article/pii/S0921889023001720>
- 2: ***, , , <https://www.sciencedirect.com/science/article/pii/S1569843225002377>
- 3: ***, , , <https://px4.io/>
- 4: Artymiak, Jacek, LibreOffice Calc Functions and Formulas Tips, 2011
- 5: ***, , , <https://docs.holybro.com/autopilot/pixhawk-6x/overview>
- 6: , , , <https://github.com/betaflight/betaflight>
- 7: , , , <https://github.com/espressif/esp-drone>
- 8: ***, , , <https://stickmanphysics.com/physics-of-drones/>
- 9: ***, , , <https://www.motioncontroltips.com/what-are-coreless-dc-motors/>
- 10: Karl J. Astrom, Tore Hagglund, PID Controllers, 2nd Edition , 1995
- 11: Jacques F. Smuts, A New Perspective on the Tuning, Stability, and Benefits of,
- 12: , , , <https://learn.sparkfun.com/tutorials/accelerometer-basics/all>
- 13: Mark Pedley, Tilt Sensing Using a Three-Axis Accelerometer,
- 14: Oliver J. Woodman, An introduction to inertial navigation,
- 15: ***, ,
- 16: ***, ,
- 17: Dominik Honegger, Lorenz Meier, Petri Tanskanen and Marc Pollefeys, An Open Source and Open Hardware Embedded Metric Optical Flow,
- 18: ***, ,
- 19: ***, , , <https://docs.espressif.com/projects/esp-idf/en/v6.1/esp32s3/about.html#id1>
- 20: ***, , , <https://reveng.sourceforge.io/crc-catalogue/16.htm>
- 21: ***, , , https://github.com/PX4/PX4-ECL/blob/master/EKF/sensor_range_finder.hpp
- 22: ***, , , <https://discuss.ardupilot.org/t/tilt-compensation-for-lidar/21003>
- 23: ***, , , <https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/ledc.html>
- 24: ***, , , <https://fishpepper.de/2016/09/04/betaflight-setup-for-brushed-flight-controllers/>
- 25: ***, , , <https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/wifi-driver/wifi-performance-and-power-save.html#esp32-s3-wi-fi-power-saving-mode>
- 26: ***, , , https://www.agisoft.com/pdf/tips_and_tricks/Image%20Capture%20Tips%20-%20Equipment%20and%20Shooting%20Scenarios.pdf
- 27: ***, , , <https://docs.espressif.com/projects/esp-idf/en/v4.4.8/esp32/api-reference/peripherals/adc.html>
- 28: ***, , , <https://docs.espressif.com/projects/esp-hardware-design-guidelines/en/latest/esp32s3/schematic-checklist.html>